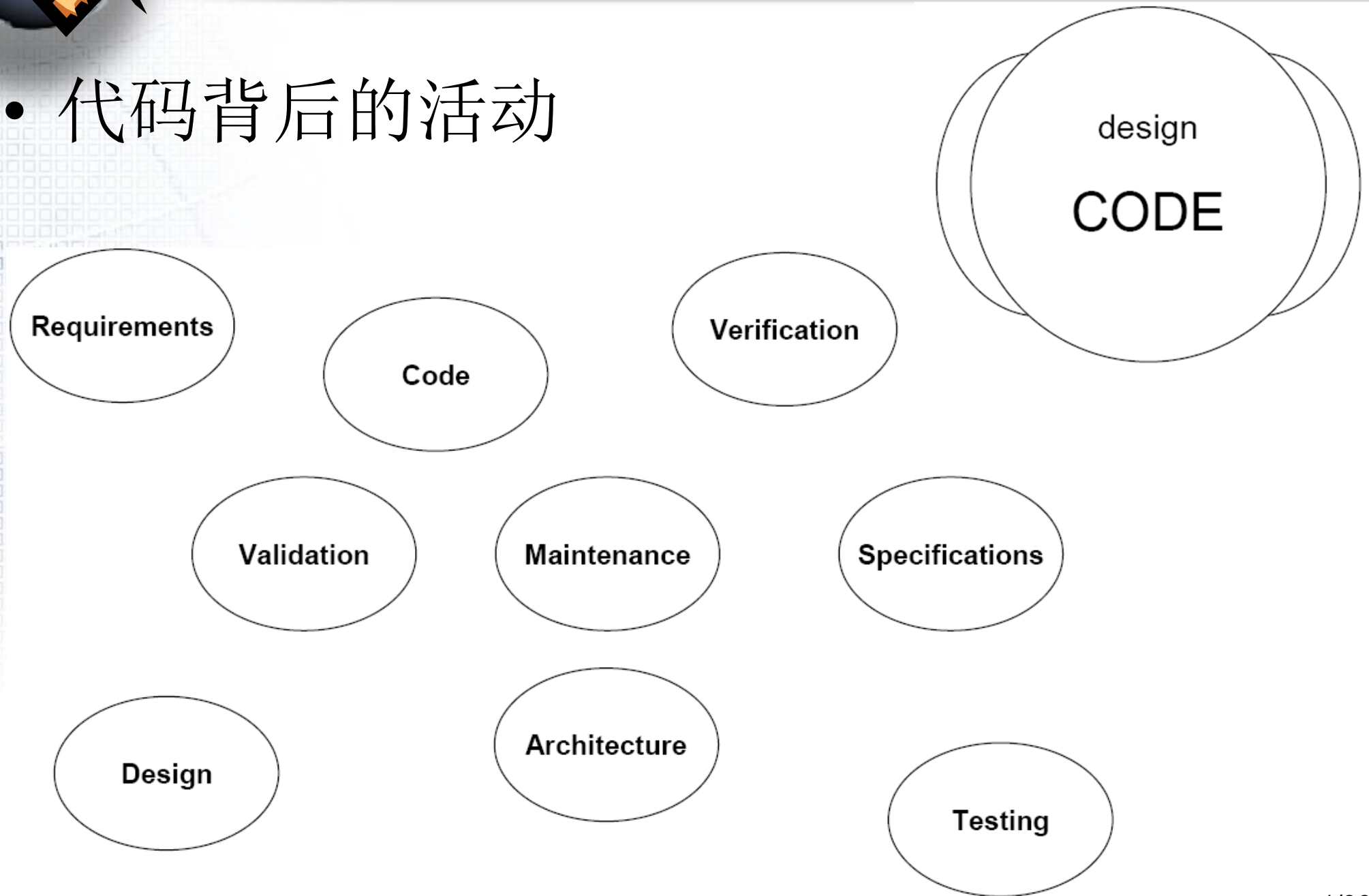
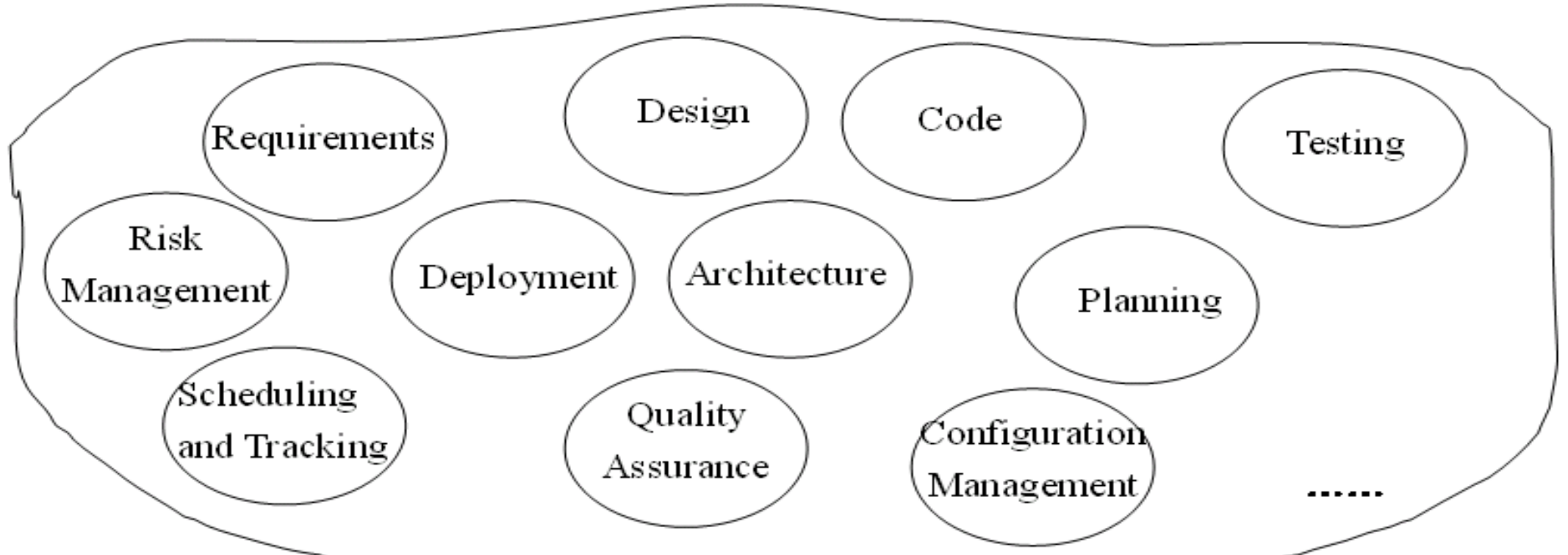




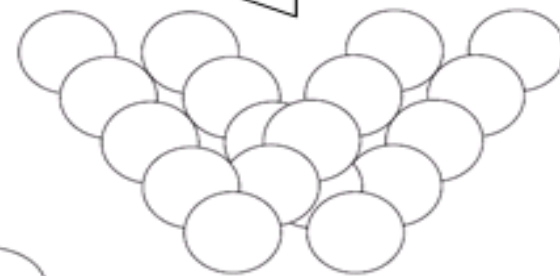
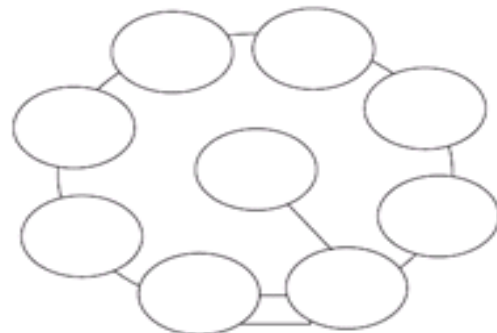
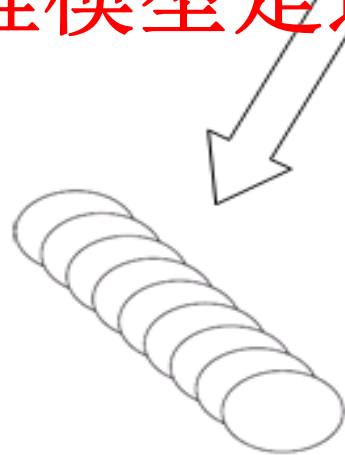
一个软件工程师的视角

- 代码背后的活动





过程模型是过程的组织策略，是过程抽象表示





Chapter 3 软件过程模型

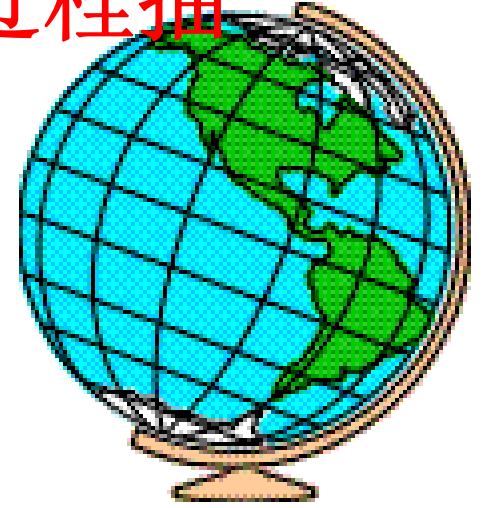
- 3.1 过程模型概念
- 3.2 线形顺序模型系列
- 3.3 演进模型系列
- 3.4 其它模型系列
- 3.5 过程模型的选择



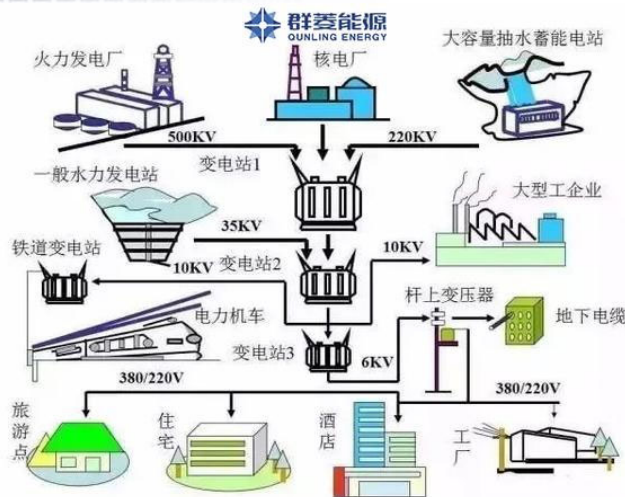


3.1 过程模型概念(1/4)

过程模型是过程的组织策略，是过程抽象表示



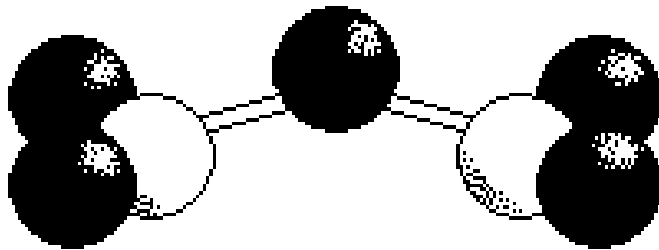
- 为什么需要模型？
 - 模型帮助我们解释事物是如何工作
 - 指导做事的步骤
 - 模型能够拓宽我们的视野(抽象)





3.1 过程模型概念(2/4)

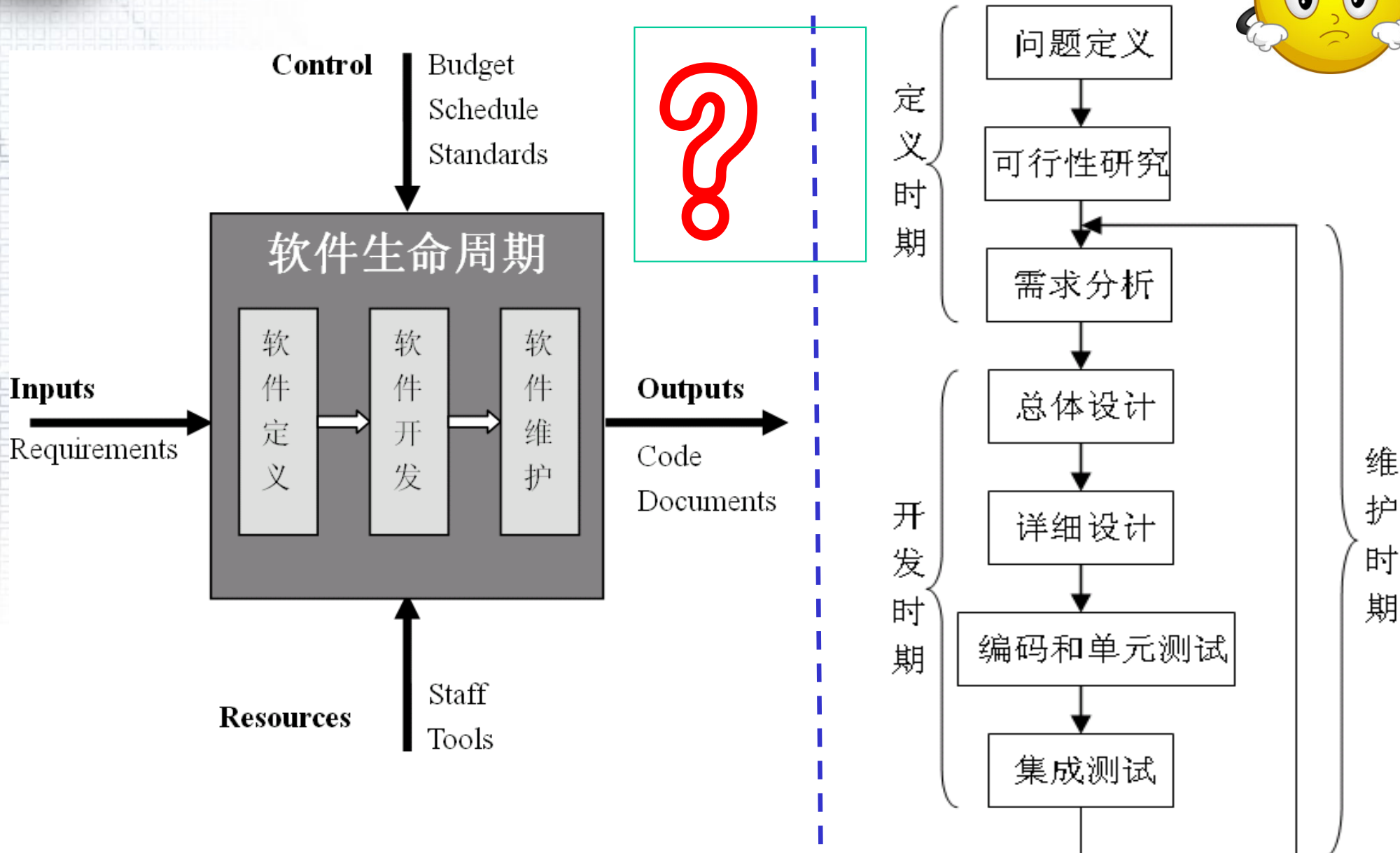
- 软件过程模型
 - 过程模型帮助我们更好地理解软件开发
 - 模型有助于理解与沟通
 - 模型有助于产生一致文档
 - 自动代码生成
 - 模型有助于理论研究





3.1 过程模型概念(3/4)

著名的传统“软件生命周期”理论





3.1 过程模型概念(4/4)

- 经典模型

- Linear Sequential Model
- Waterfall Model
- V Model
- Department of Defense Model
- RAD Model
- The Circular Model

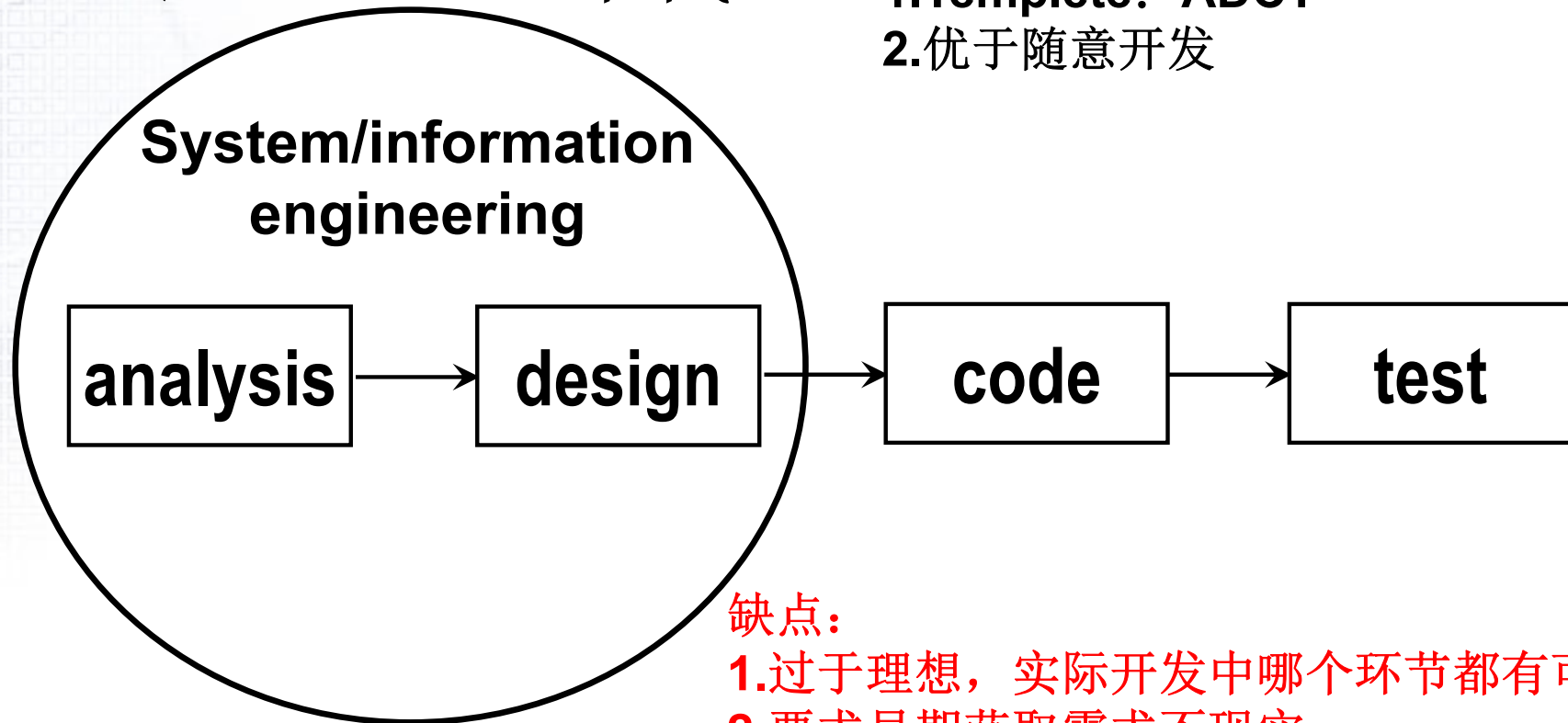
- Prototyping Model
- Build-and-Fix Model
- Incremental Model
- Spiral Model
- XP Model
- RUP Model
- CBD Assembly Model
- Formal Methods Model
- IDEAL Model



3.2 线形顺序模型系列(1/7)

- 线性顺序模型

—20世纪60~70年代



优点:

1. **Template: ADCT**
2. 优于随意开发

缺点:

1. 过于理想，实际开发中哪个环节都有可能出错
2. 要求早期获取需求不现实
3. 用户与开发方无交互
4. 无并行



3.2 线形顺序模型系列(2/7)

70年代提出，80年代早期

• 瀑布模型

特点:

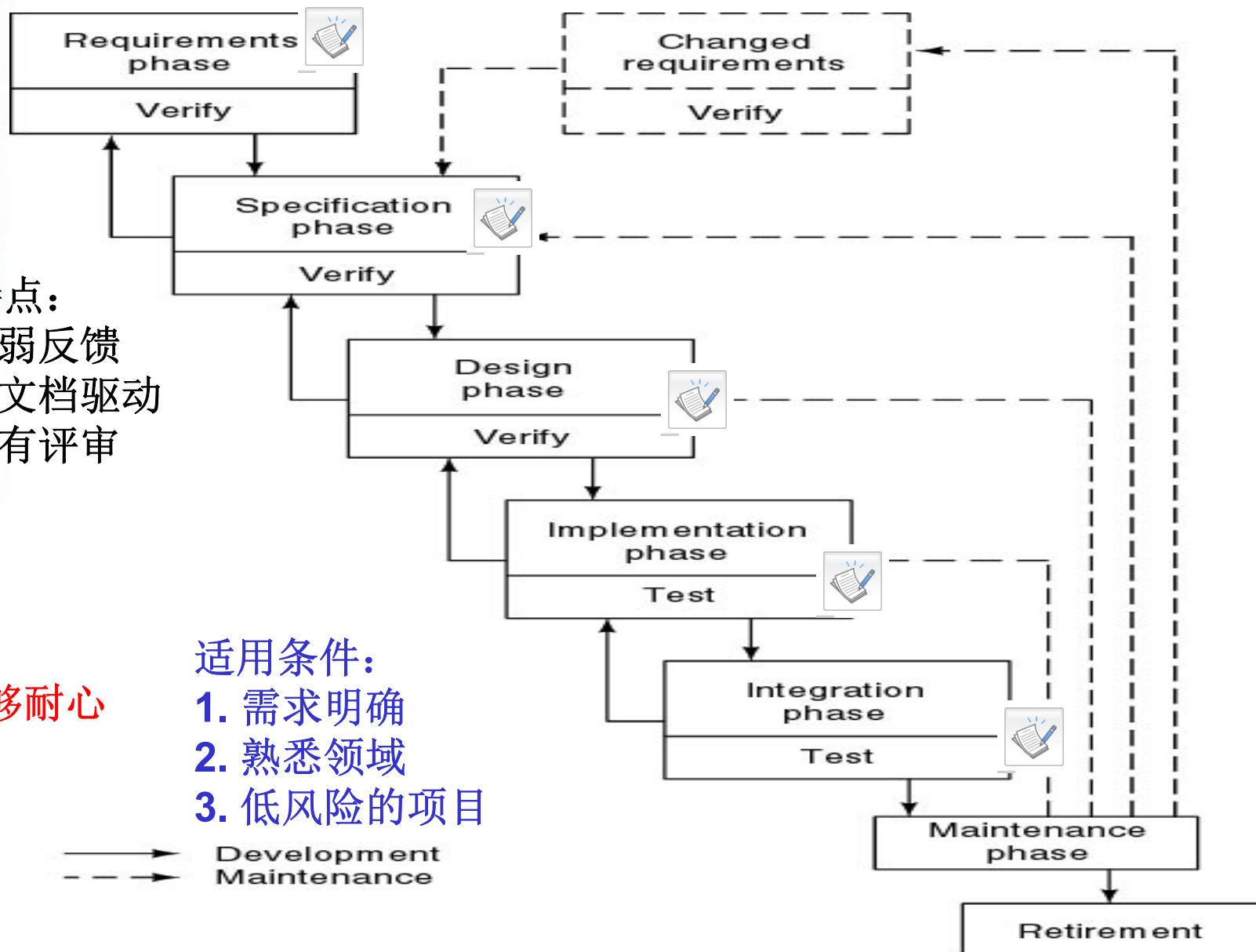
1. 弱反馈
2. 文档驱动
3. 有评审

缺点:

1. 依赖文档
2. 问题堆积
3. 没有并行
4. 用户需要足够耐心

适用条件:

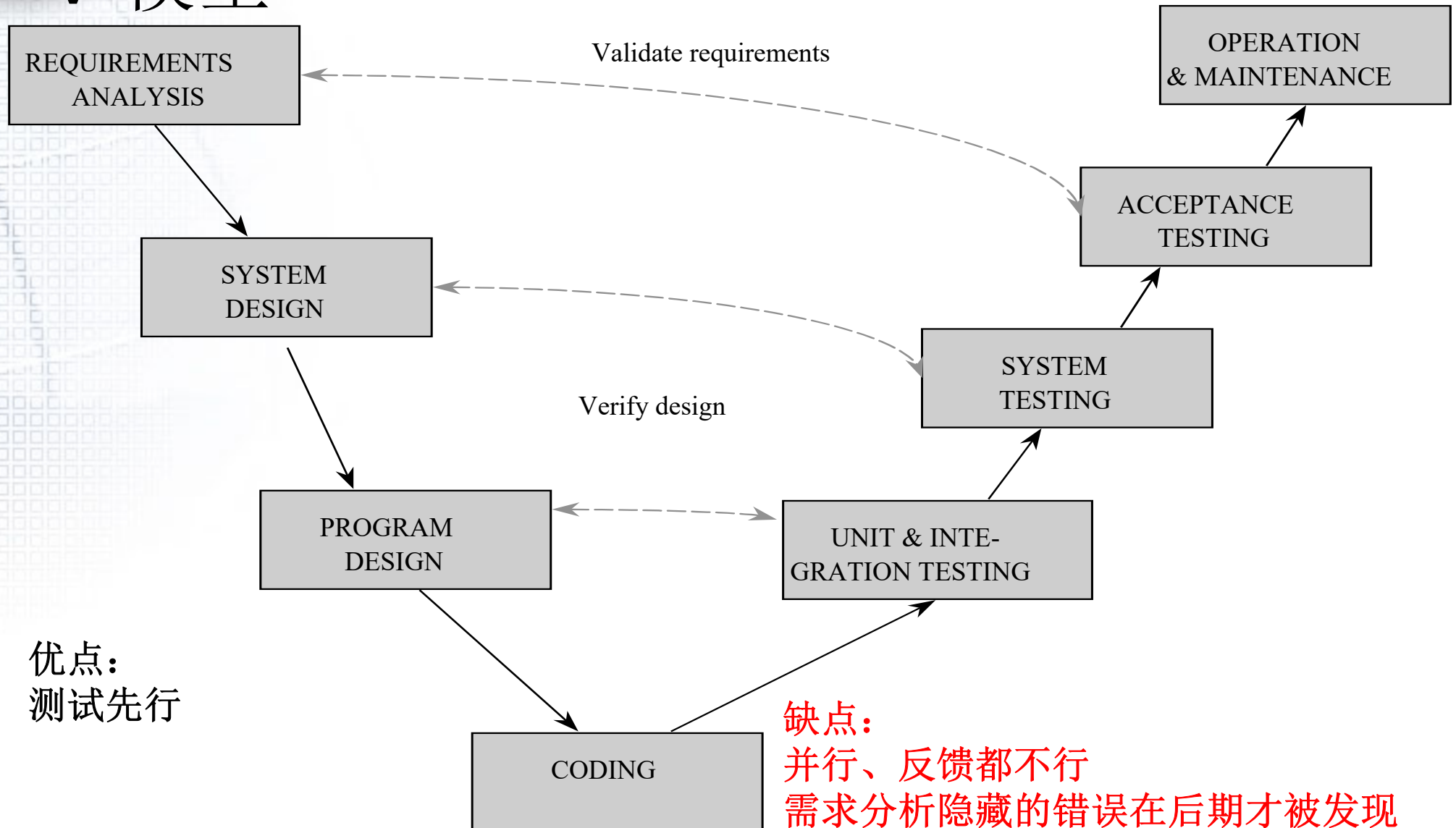
1. 需求明确
2. 熟悉领域
3. 低风险的项目





3.2 线形顺序模型系列(3/7)

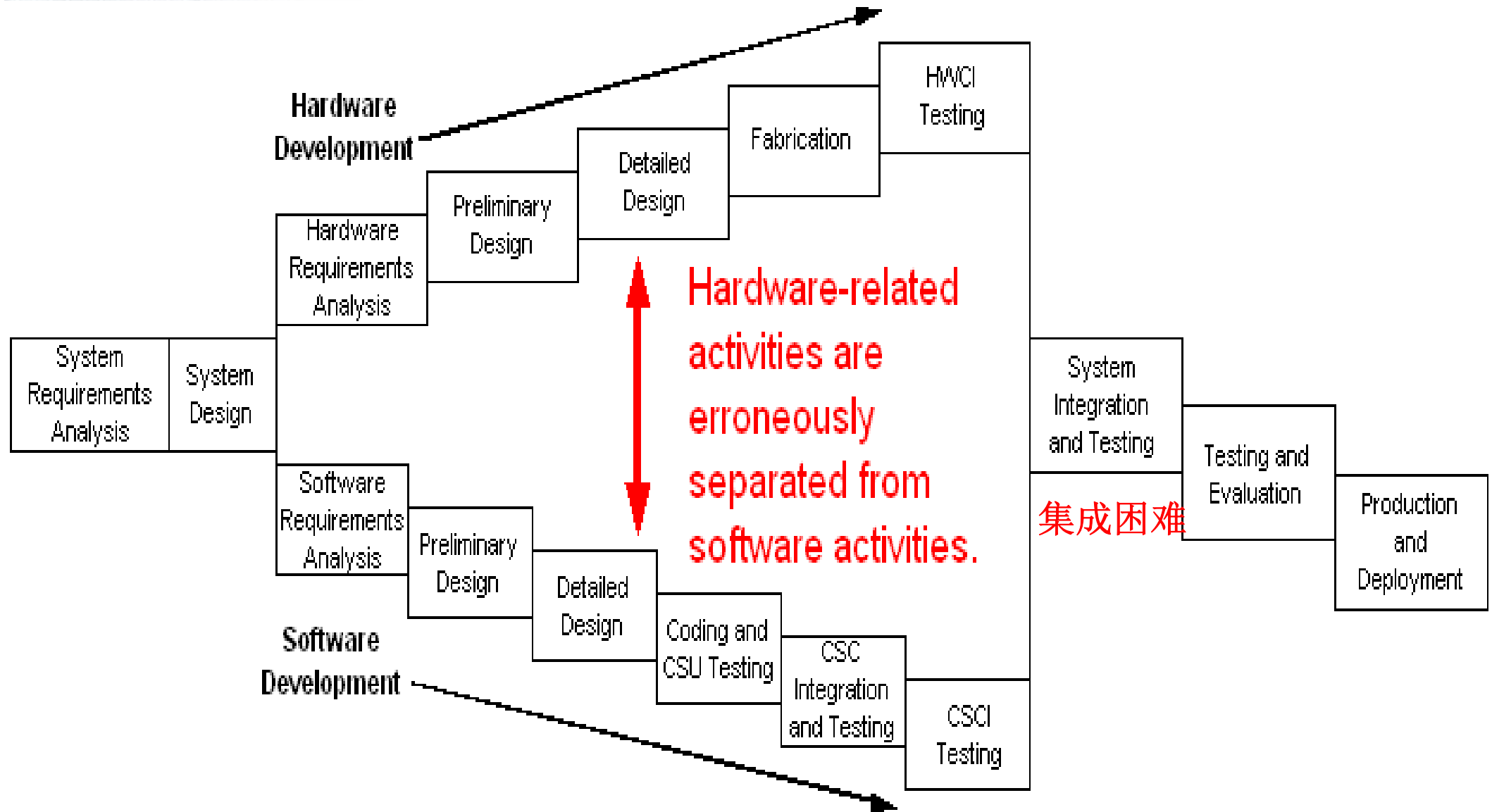
• V 模型





3.2 线形顺序模型系列(4/7)

- 国防部模型

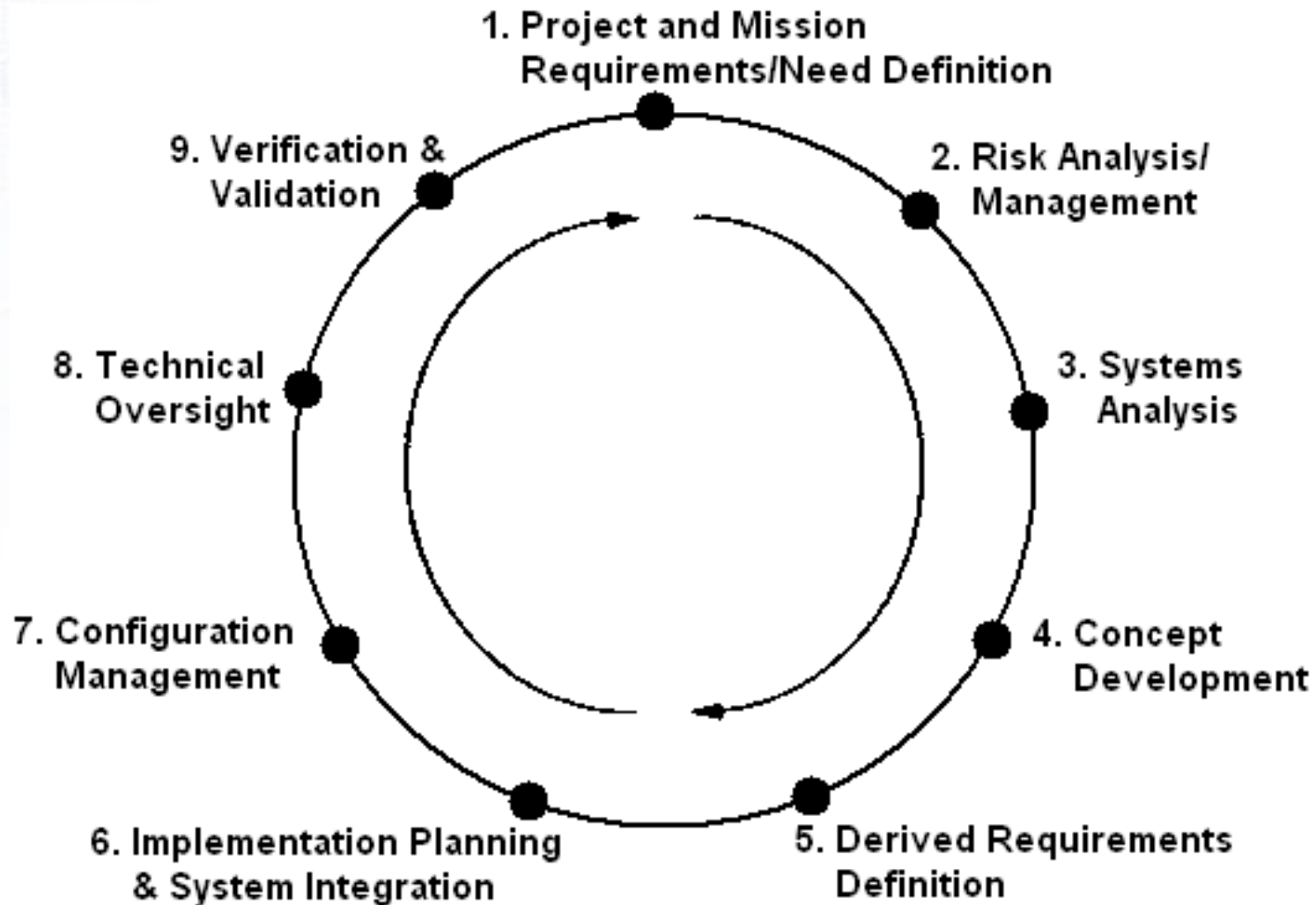




3.2 线形顺序模型系列(5/7)

增加了辅助性活动

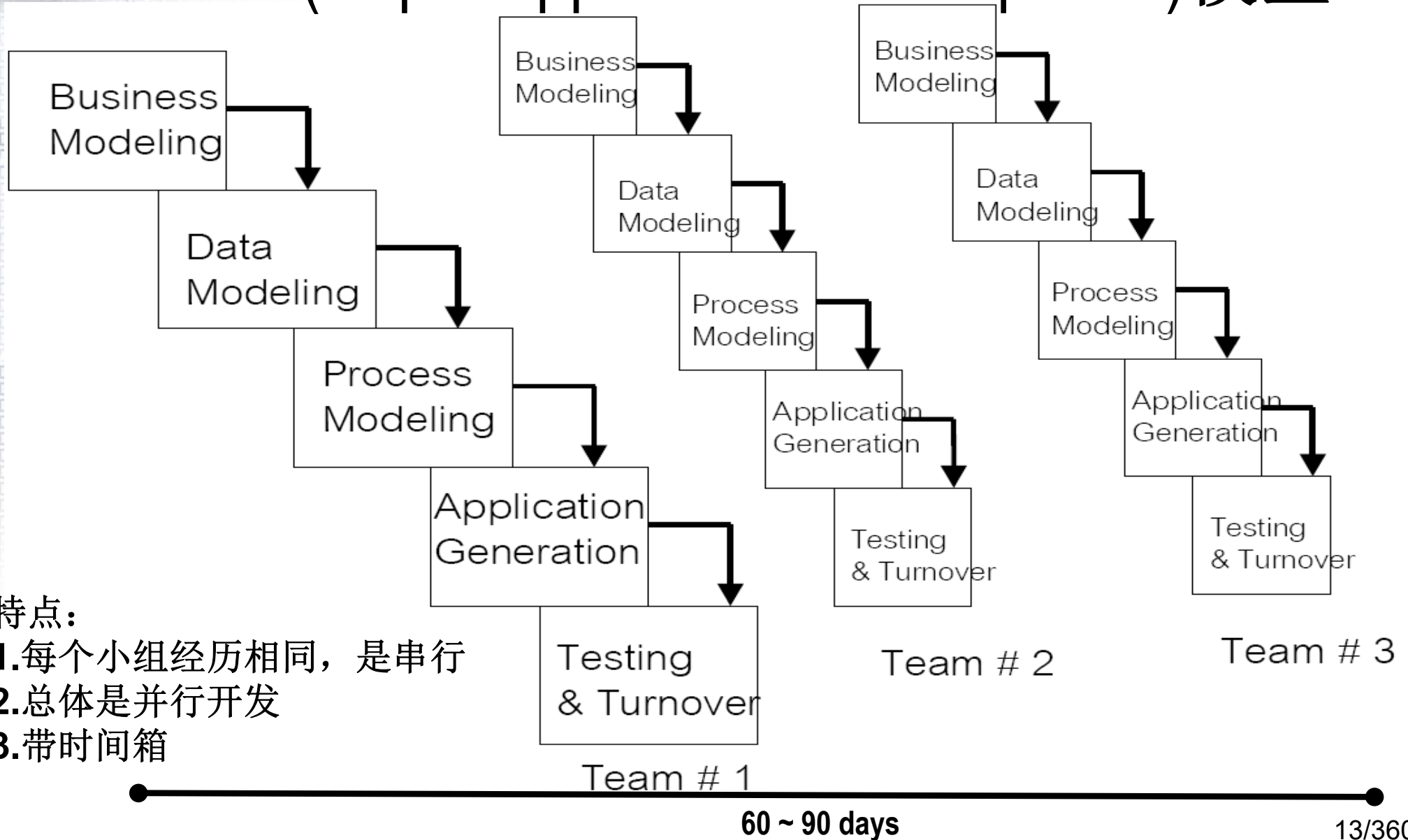
- 圆形模型





3.2 线形顺序模型系列(6/7)

- RAD (Rapid Application Development)模型





3.2 线形顺序模型系列(7/7)

- RAD模型优点:

- 开发速度快
- 鼓励复用

适用条件:

1. 需求明确完整
2. 模块化程度高
3. 技术成熟
4. 低风险的项目
5. 鼓励复用

- RAD模型缺点:

- 如果一个系统难以适当模块化，则很难组织并行开发
- 不鼓励需求变更
- 缺乏类似经验时，很难快速设计系统
- 不适合高技术风险项目
- 不适合规模较大的项目



线性模型小结

- L
- WM
- V
- C
- DoD
- RAD



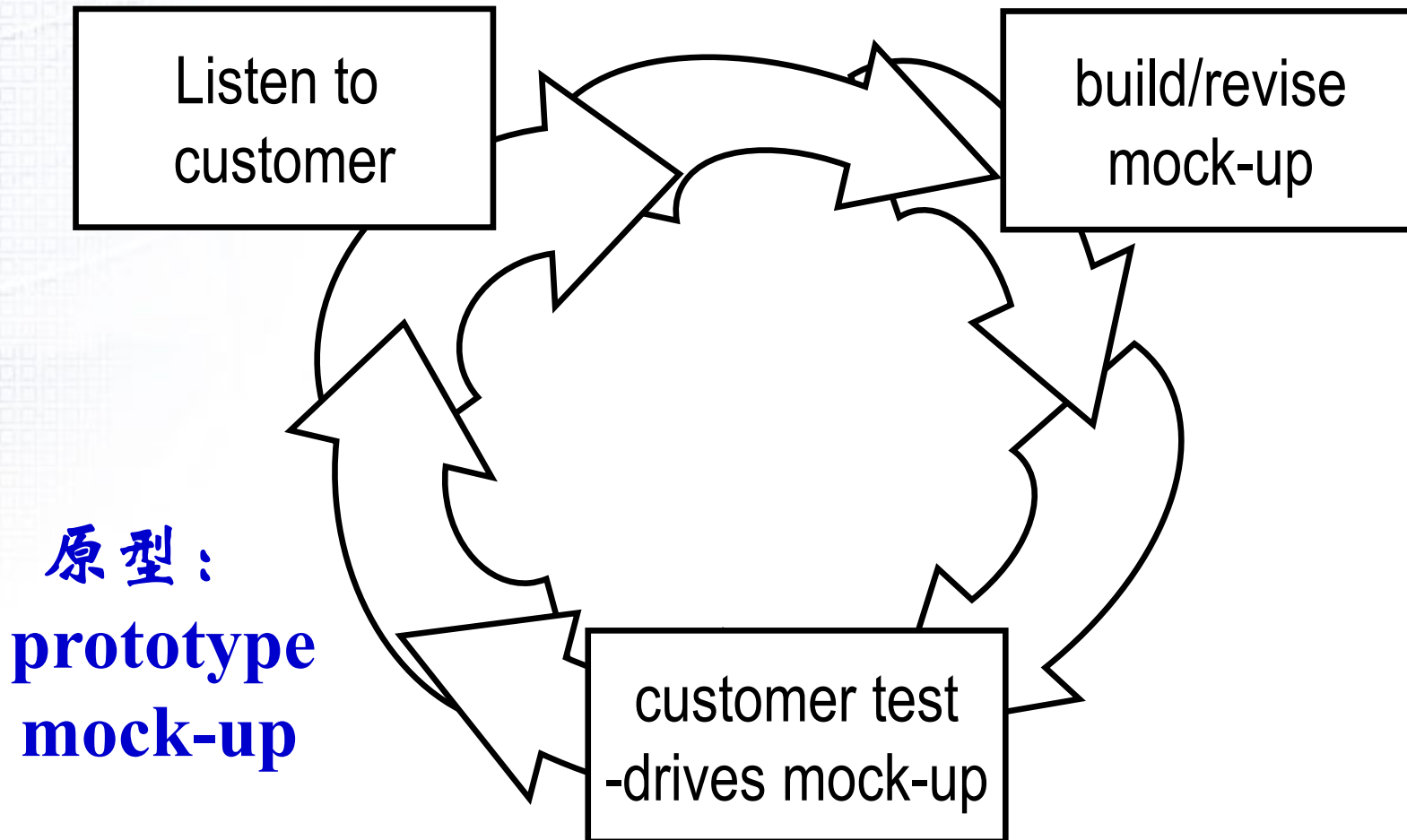
3.3 演进模型系列(1/17)

- 迭代式开发
 - 工作版本



3.3 演进模型系列(2/17)

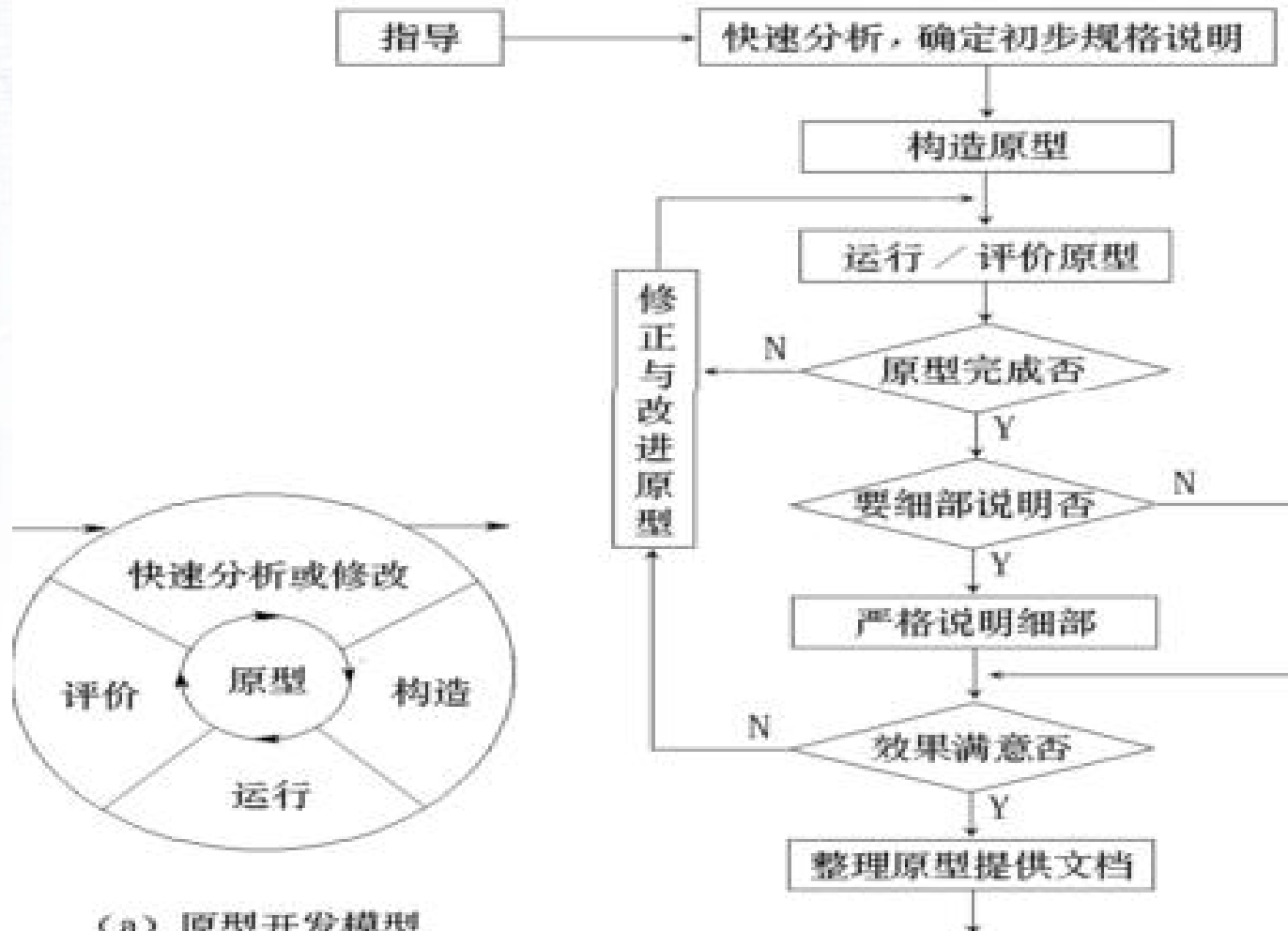
- 原型模型(1/2)





3.3 演进模型系列(3/17)

• 原型模型(2/2)



优点:
早期获取需求
降低需求风险

缺点:
不是真正系统

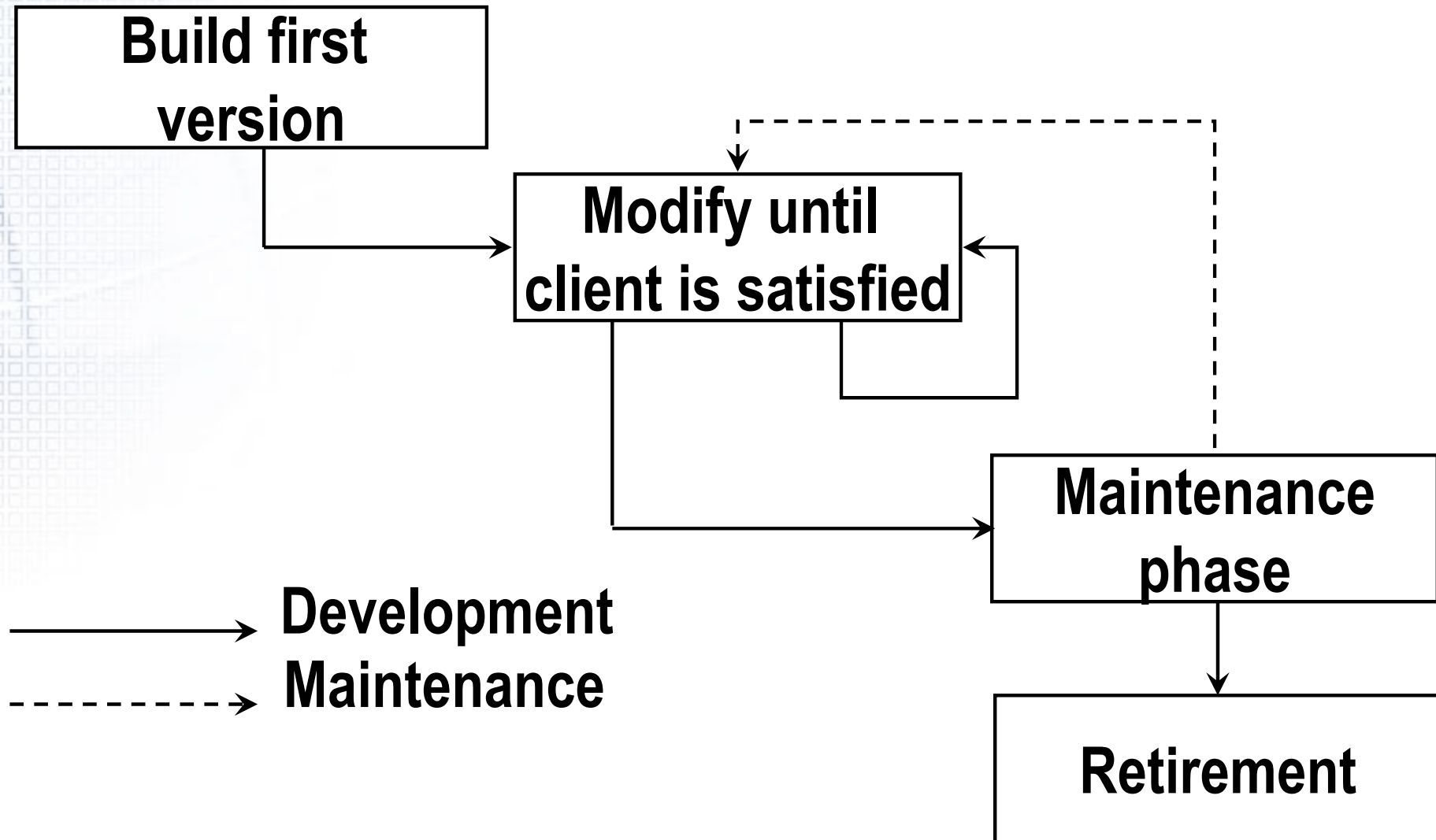
(a) 原型开发模型

(b) 模型的细化过程



3.3 演进模型系列(4/17)

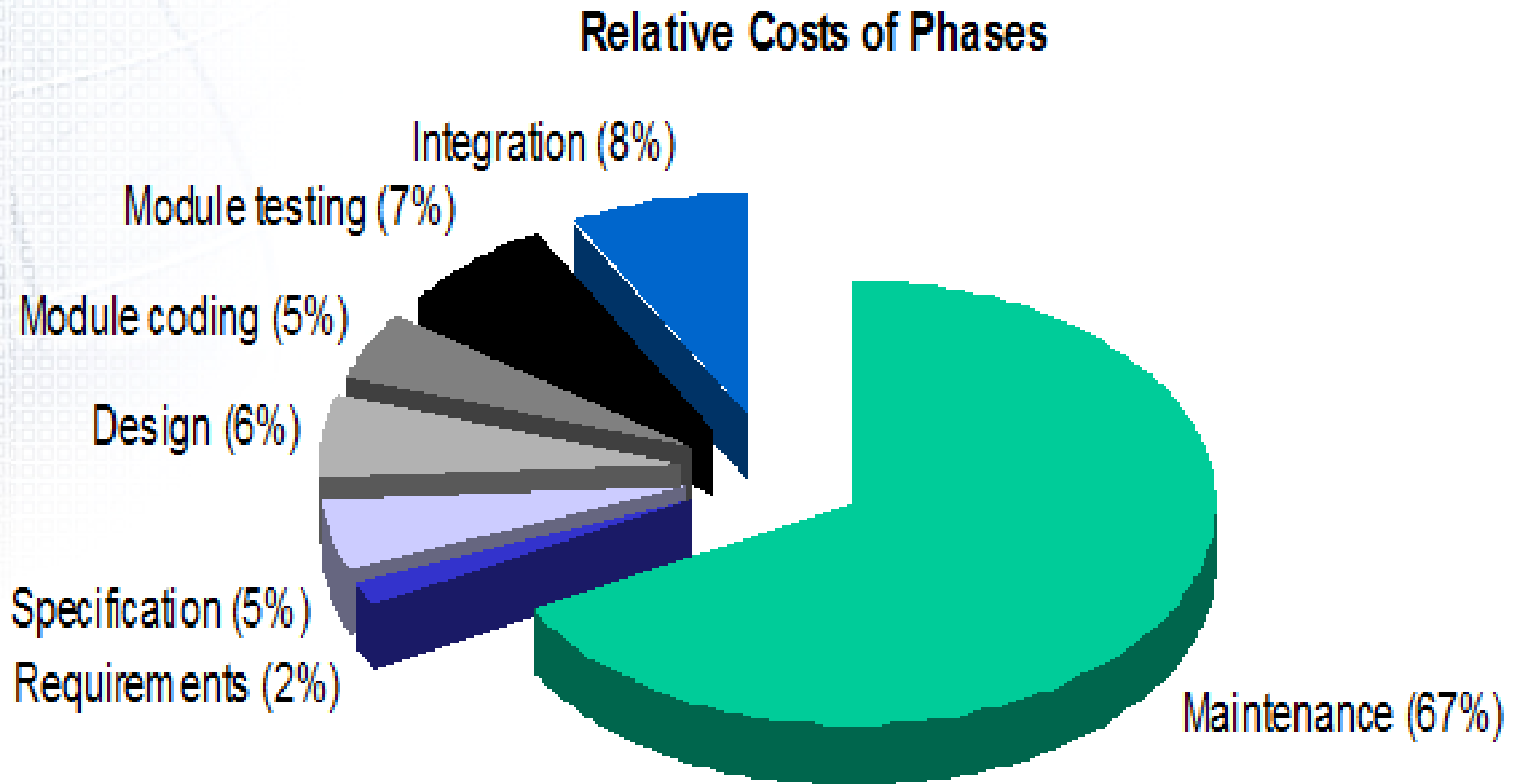
- 边建边改 Model





3.3 演进模型系列(5/17)

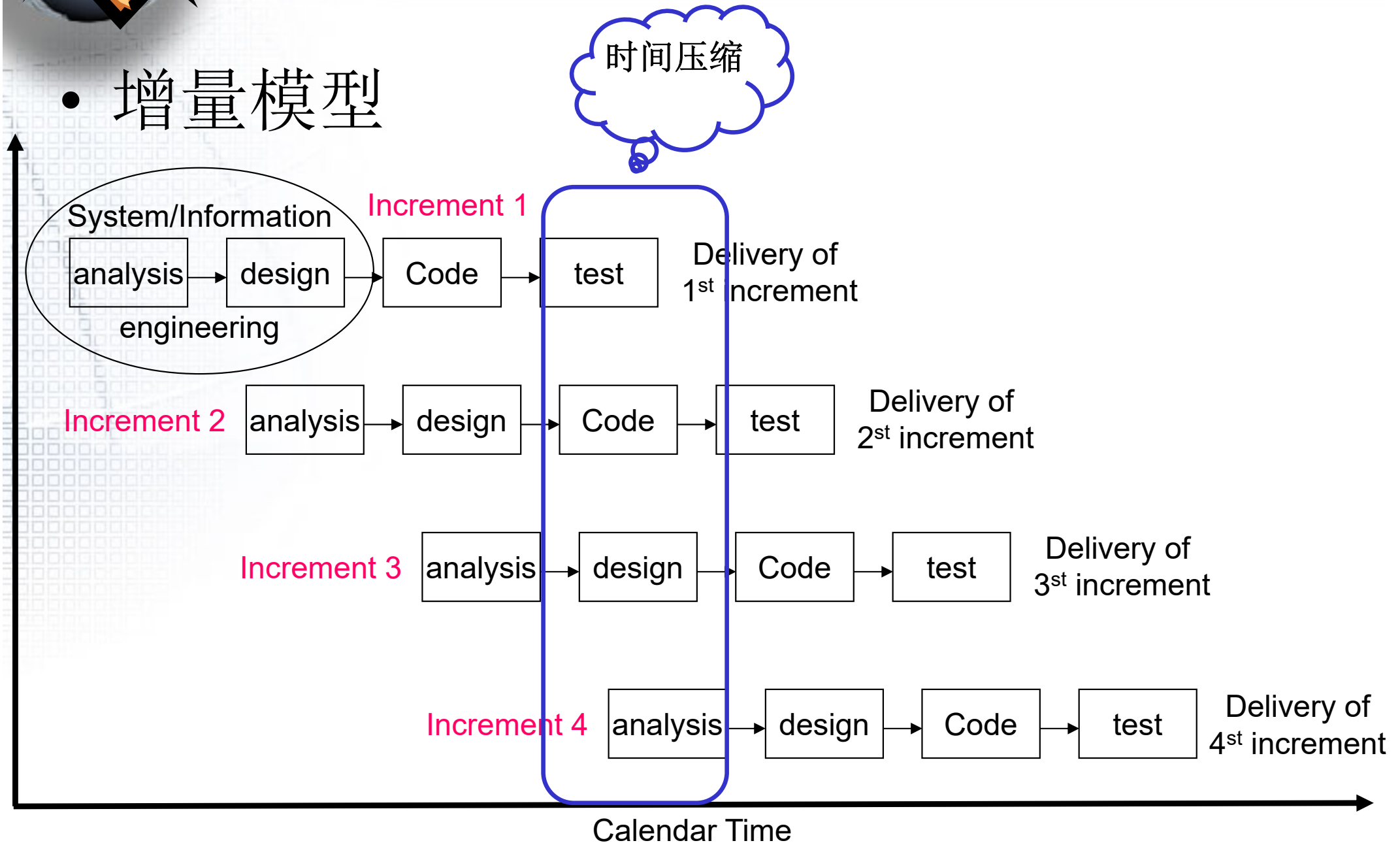
- 边建边改 Model(续)





3.3 演进模型系列(6/17)

- 增量模型





3.3 演进模型系列(7/17)

- 增量模型优点:
 - 较好的用户交互
 - 减少开发风险

缺点:

1. 系统必须能很好的被模块化
2. 集成技术要求高

项目团队要用**Delphi**语言做一个单机的宠物商店库存管理系统，初步打算使用**Paradox**数据库。估计该系统的数据库最大需要**500MB**左右的空间。但技术人员还不清楚该数据库能容纳多大的数据量-这就是一个风险。如果在项目尾声时，才发现**Paradox**难以容纳**500MB**空间，会造成项目被动的修改设计方案、编码再重新测试，造成工作量大，而且项目可能被延期。因此，项目团队应该在早期迭代中验证**Paradox**是否合适？

- 利于探索性产品的开发
 - 案例
- 应对“最后期限”的办法之一
 - 案例

适用条件:

1. 允许分阶段交付
2. 分析设计不熟悉
3. 中或高风险项目
4. 已有类库、构件库



3.3 演进模型系列(8/17)

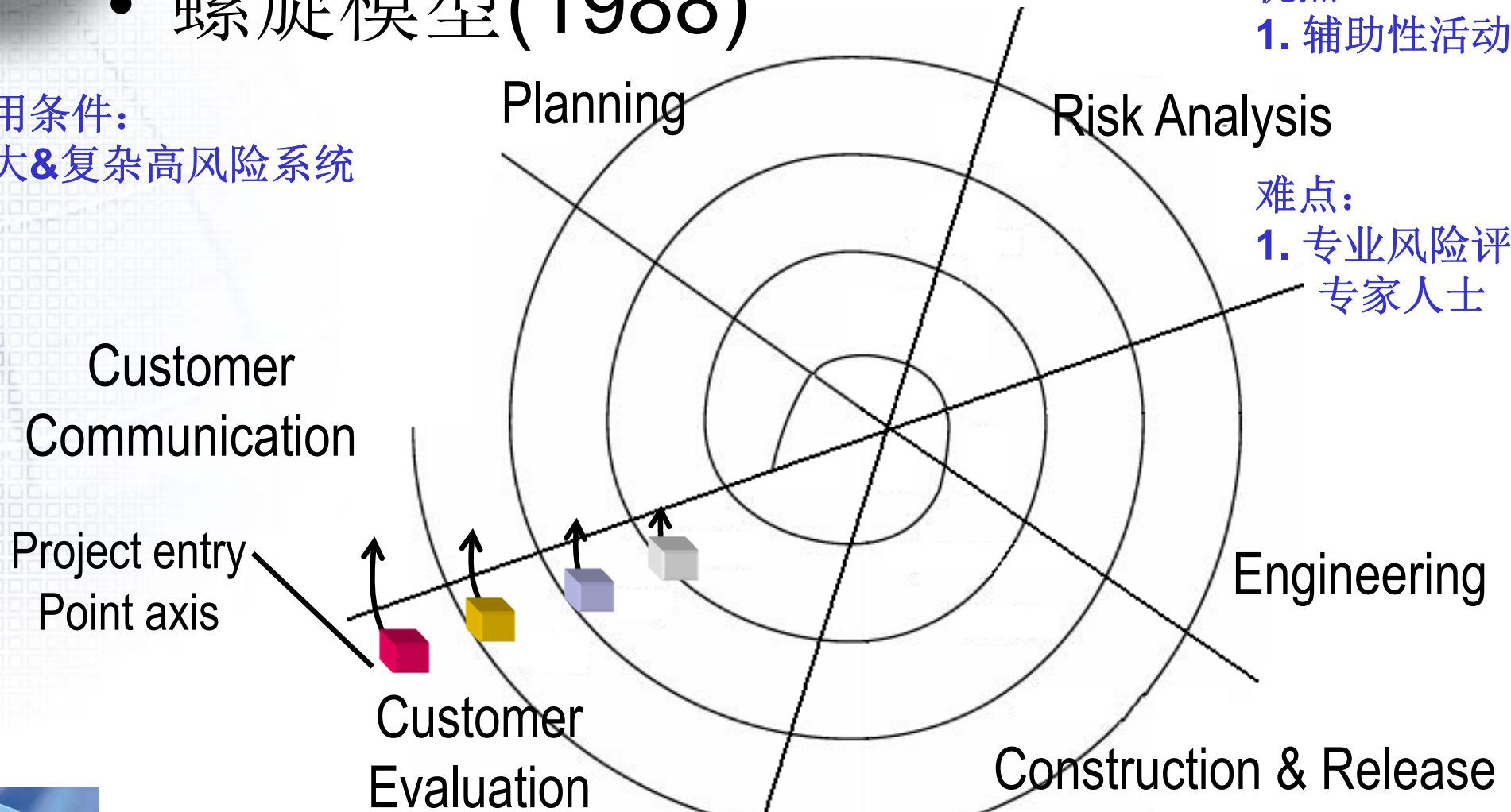
- 螺旋模型(1988)

适用条件： 庞大&复杂高风险系统

优点:

1. 辅助性活动

难点：
1. 专业风险评估
专家人士



图示：

	概念开发项目		新产品开发项目
	产品增强项目		产品维护项目





3.3 演进模型系列(9/17)

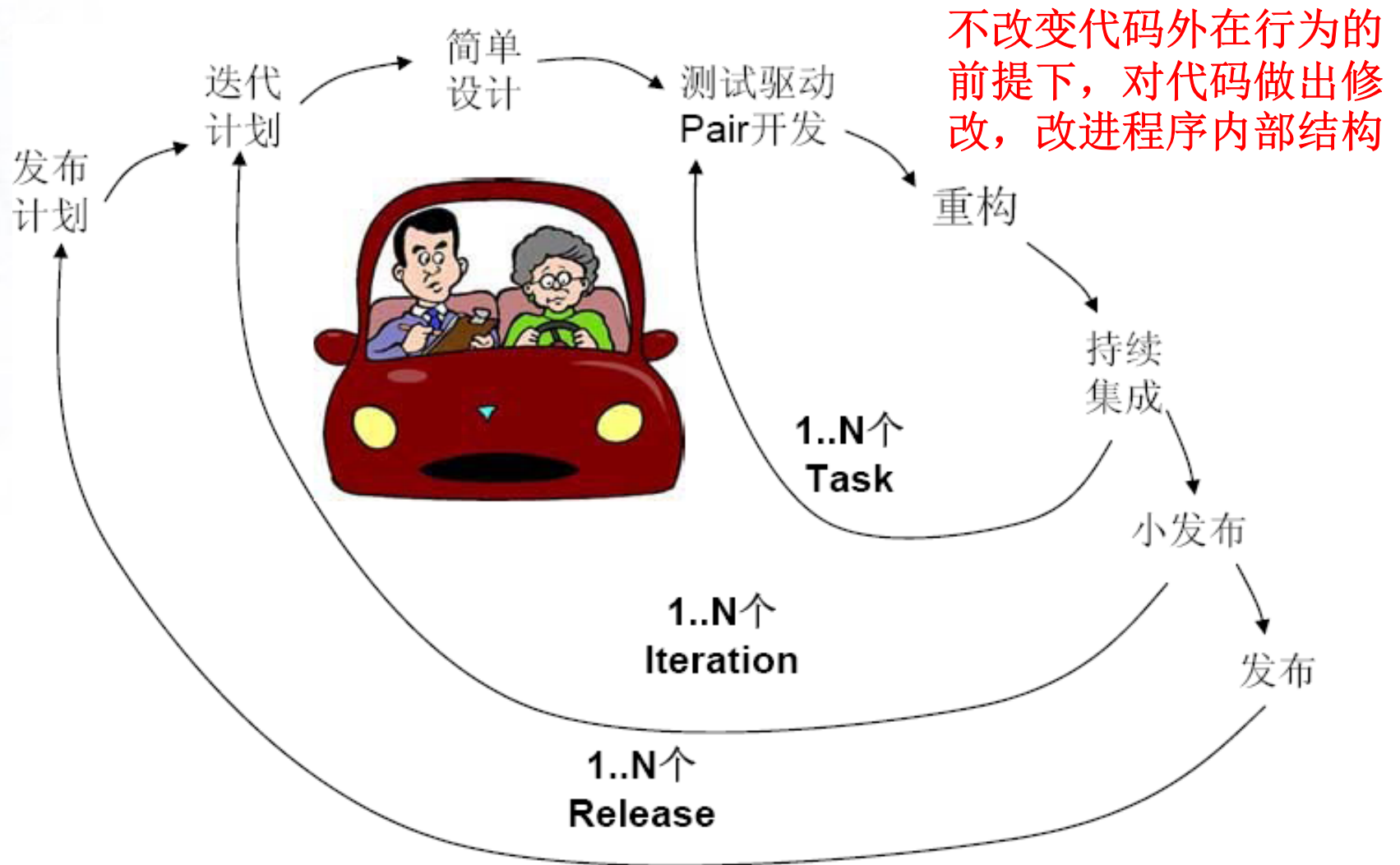
• XP 模型(eXtreme Programming)

优点:

1. 快速开发
2. 团队合作
3. 反馈

难点:

1. 缺少文档
2. 轻设计





3.3 演进模型系列(10/17)

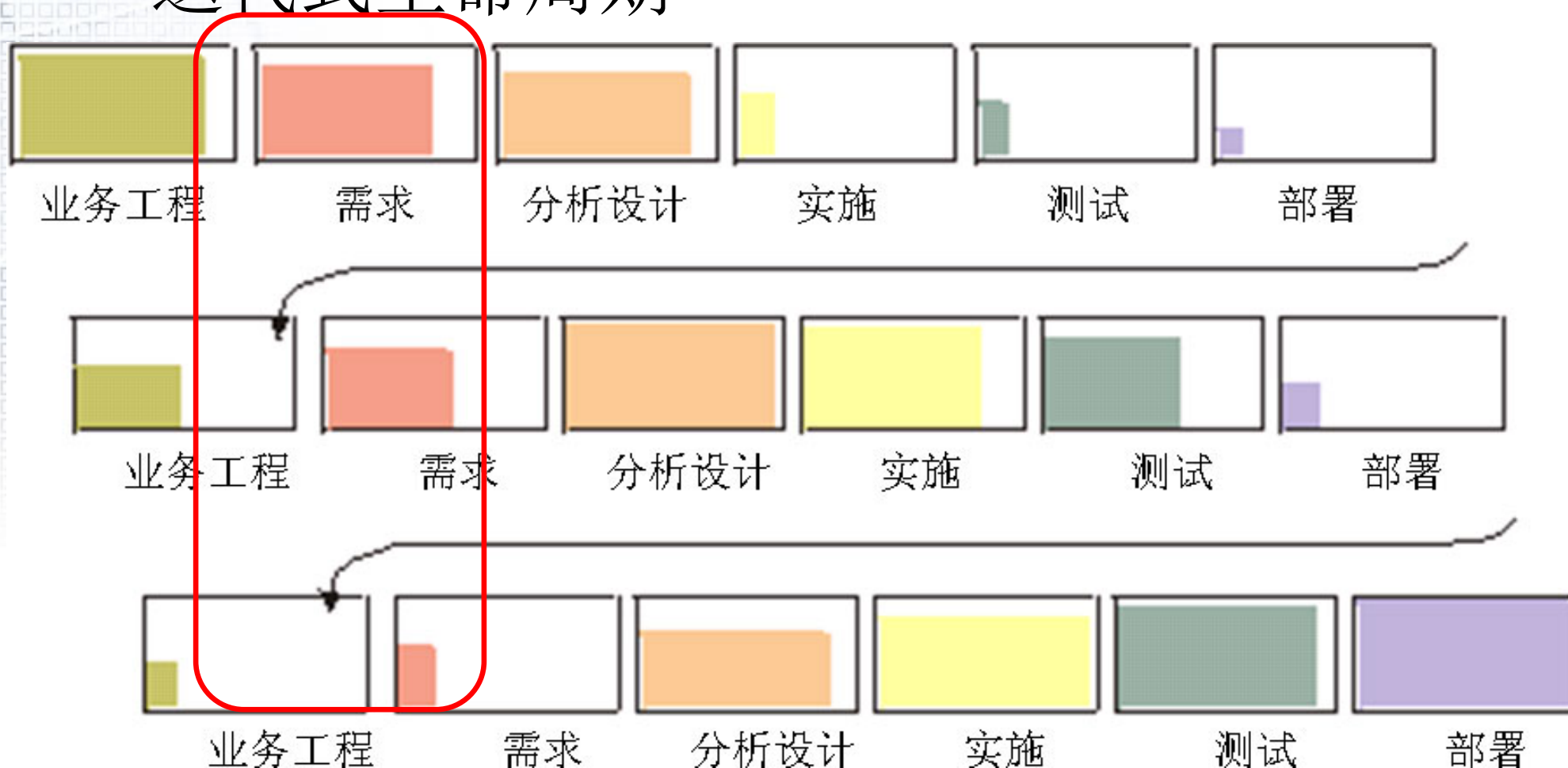
IBM RATIONAL 统一开发流程RUP

<https://www.ibm.com/developerworks/cn/rational/>



3.3 演进模型系列(11/17)

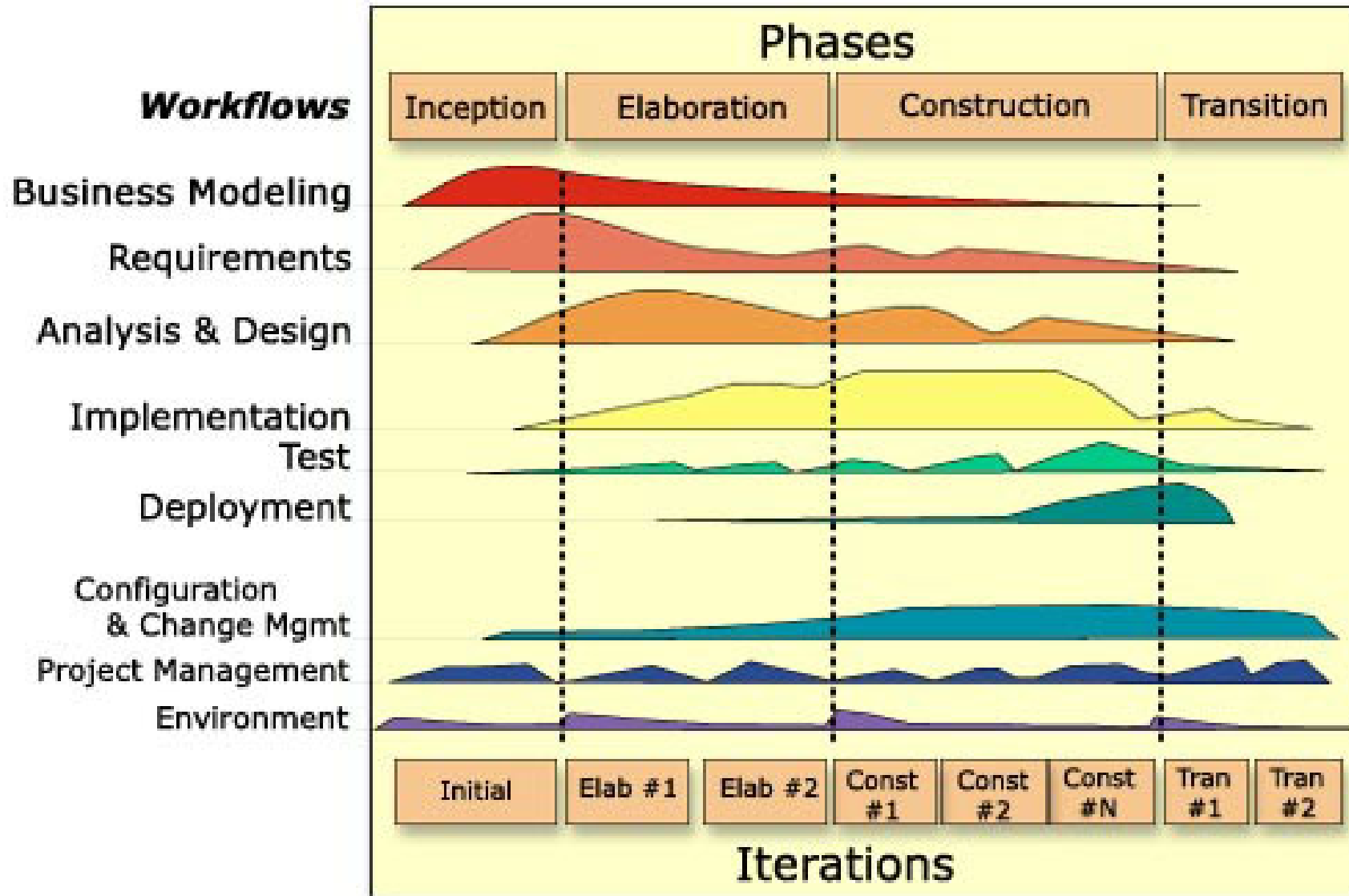
- IBM RATIONAL统一开发流程RUP
 - 迭代式生命周期





3.3 演进模型系列(12/17)

- RUP Model





3.3 演进模型系列(13/17)

- RUP Model

- RUP各阶段工作量和进度分配（中等规模项目）

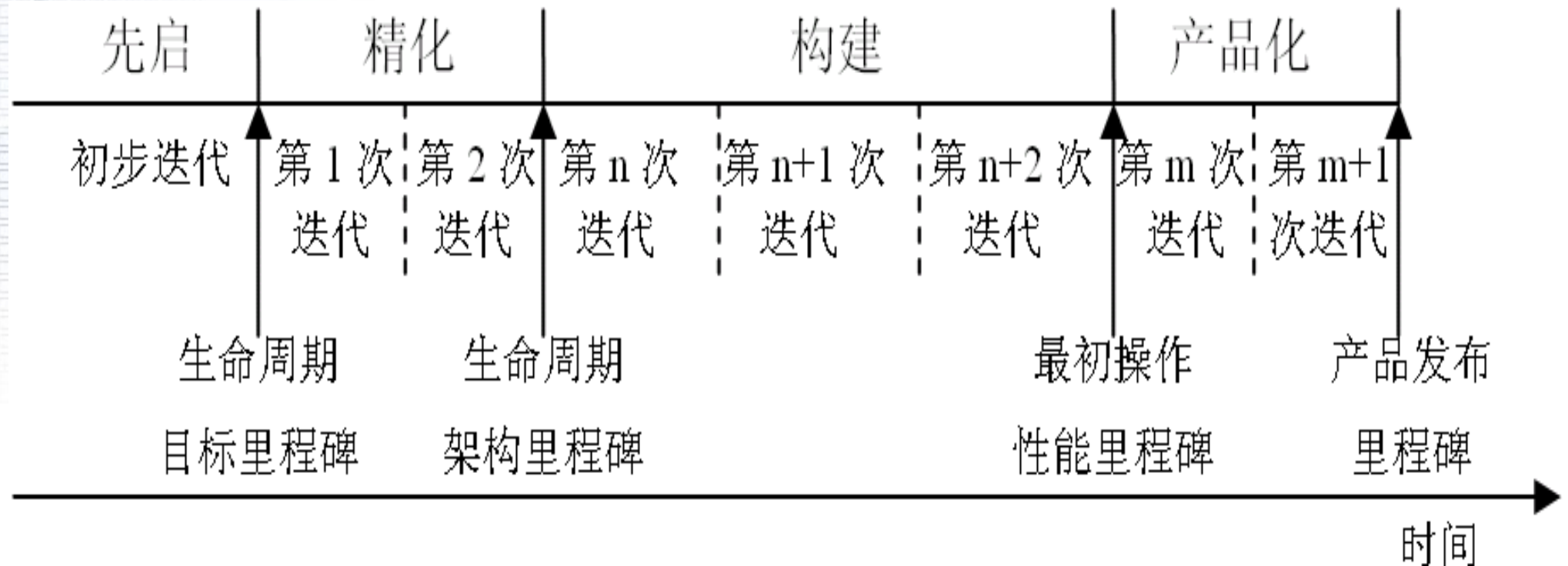
一个典型的分配方案

	先启	精化	构建	产品化
工作量	~5 %	20 %	65 %	10%
进度	10 %	30 %	50 %	10%



3.3 演进模型系列(14/17)

- RUP Model
 - 项目阶段和里程碑

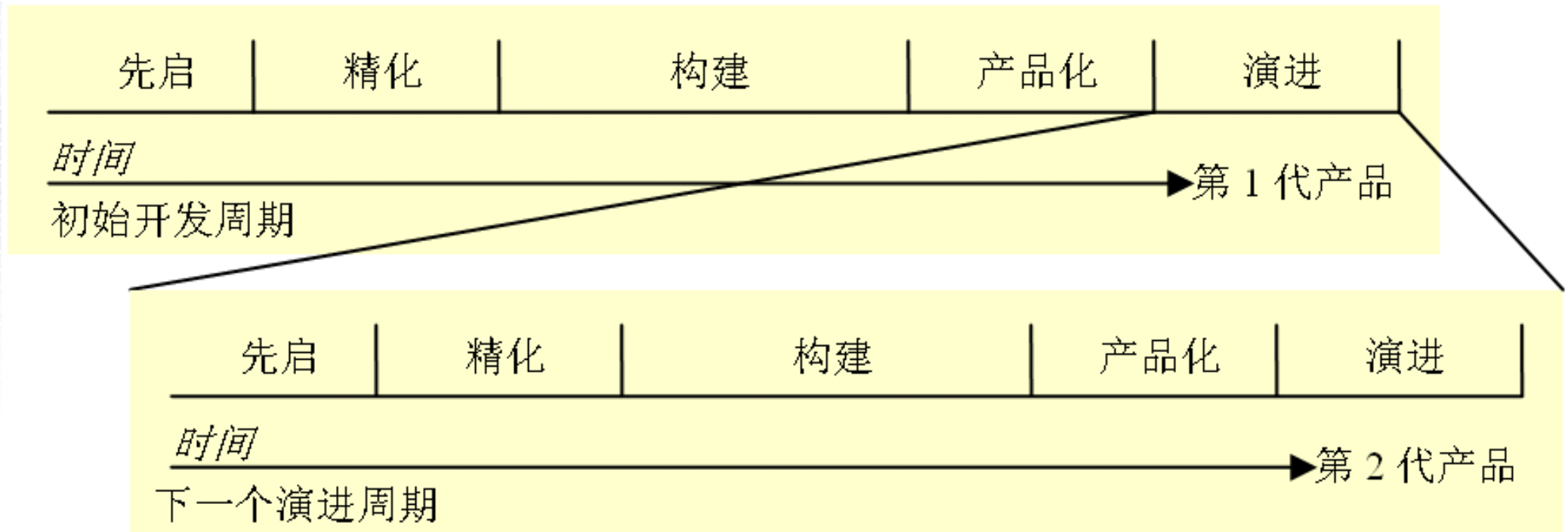




3.3 演进模型系列(15/17)

- RUP Model

- RUP开发周期和演进周期





3.3 演进模型系列(16/17)

- RUP Model 优点
 - 体系完整：迭代、里程碑、9个活动，角色、工作流程、工具
 - 理论成熟：一系列过程理论
 - 实践性强：6个最佳实践
 - 可裁剪：不死板：角色、迭代
 - 可扩充：起到指导的作用



案例

- 假设某项目团队一直在开发单机或者局域网环境下的应用系统，但从未开发过网站，没有架站经验，也不熟悉该项目所要使用的**ASP.NET**开发语言。该团队对即将开发的在线宠物商店系统的业务需求理解比较模糊。整个项目存在很多风险。好在客户规定的开发时间还比较宽裕。



演进系列模型小结

- Prototyping Model
- XP Model
- Incremental Model
- Spiral Model
- RUP Model
- Build-and-Fix Model



原型模图



XP极限模型
小铁皮房



增量模型别墅



螺旋模型



RUP天楼



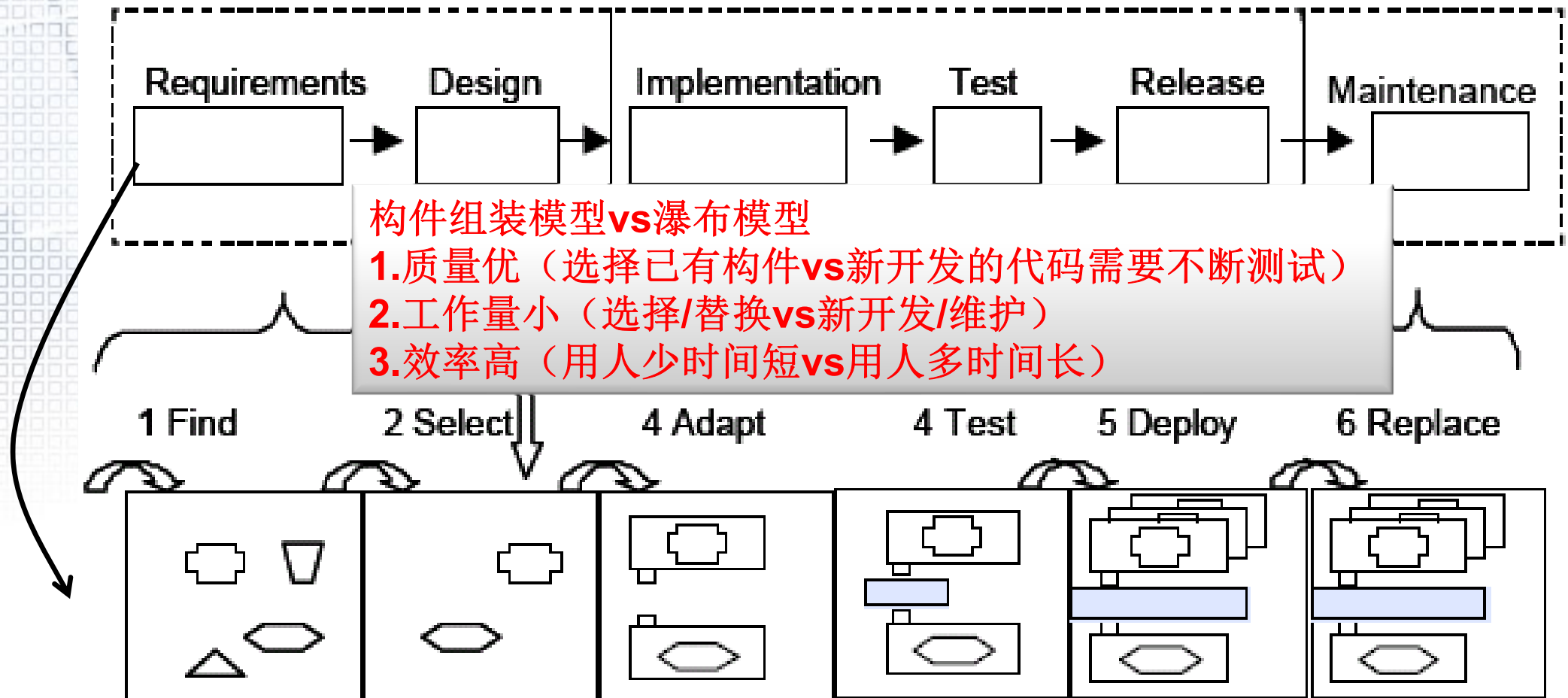
边建边改模
型破旧棚屋



3.4 其它模型系列(1/5)



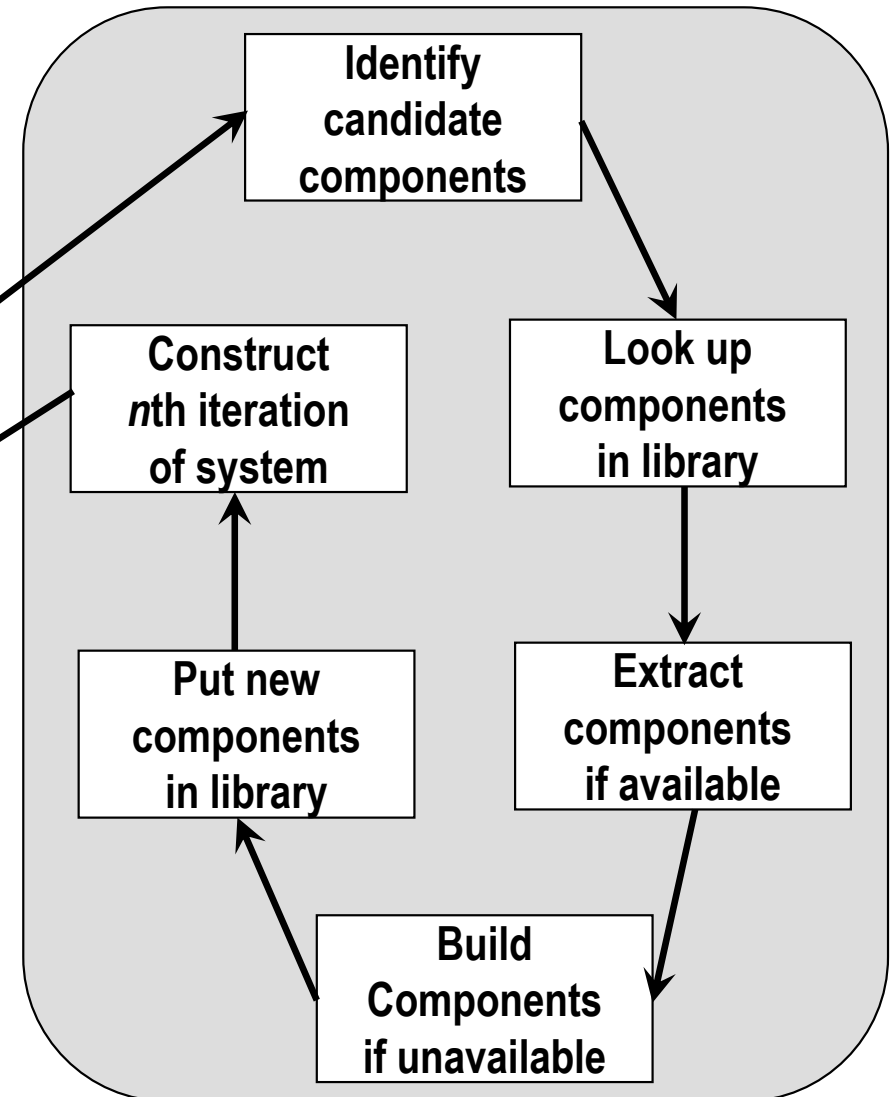
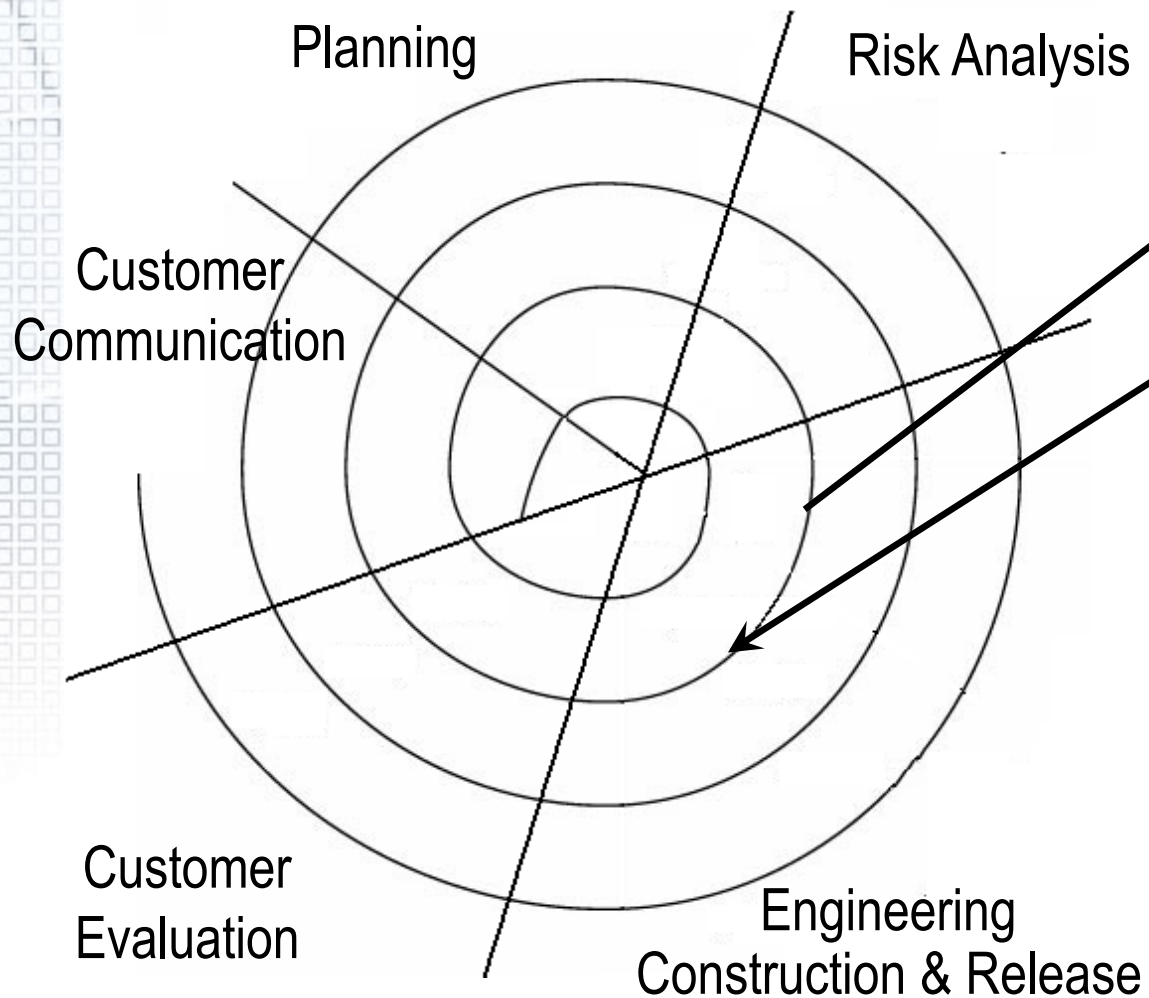
- 构件组装模型
 - 与瀑布模型对比





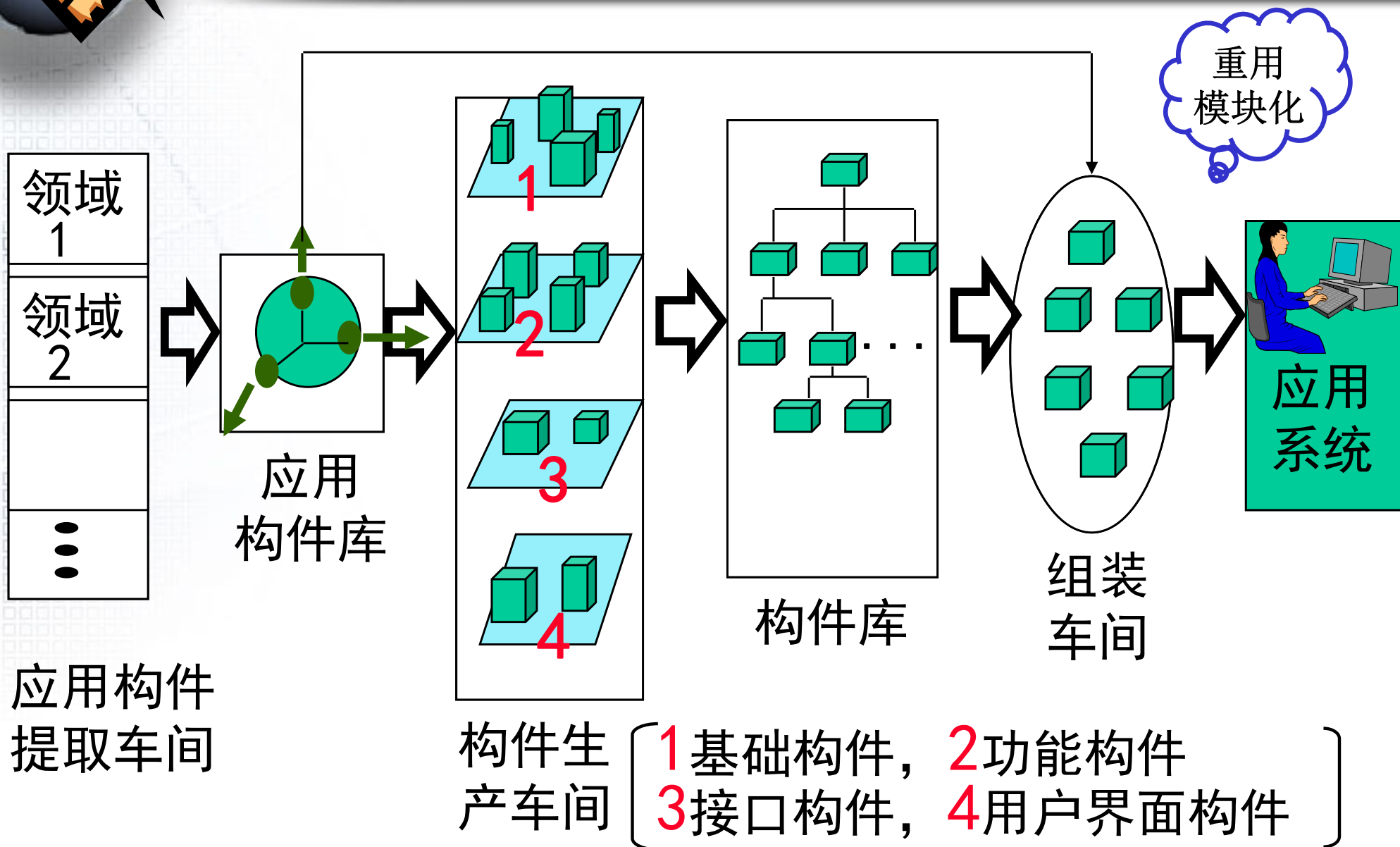
3.4 其它模型系列(2/5)

- 构件组装模型(螺旋版)





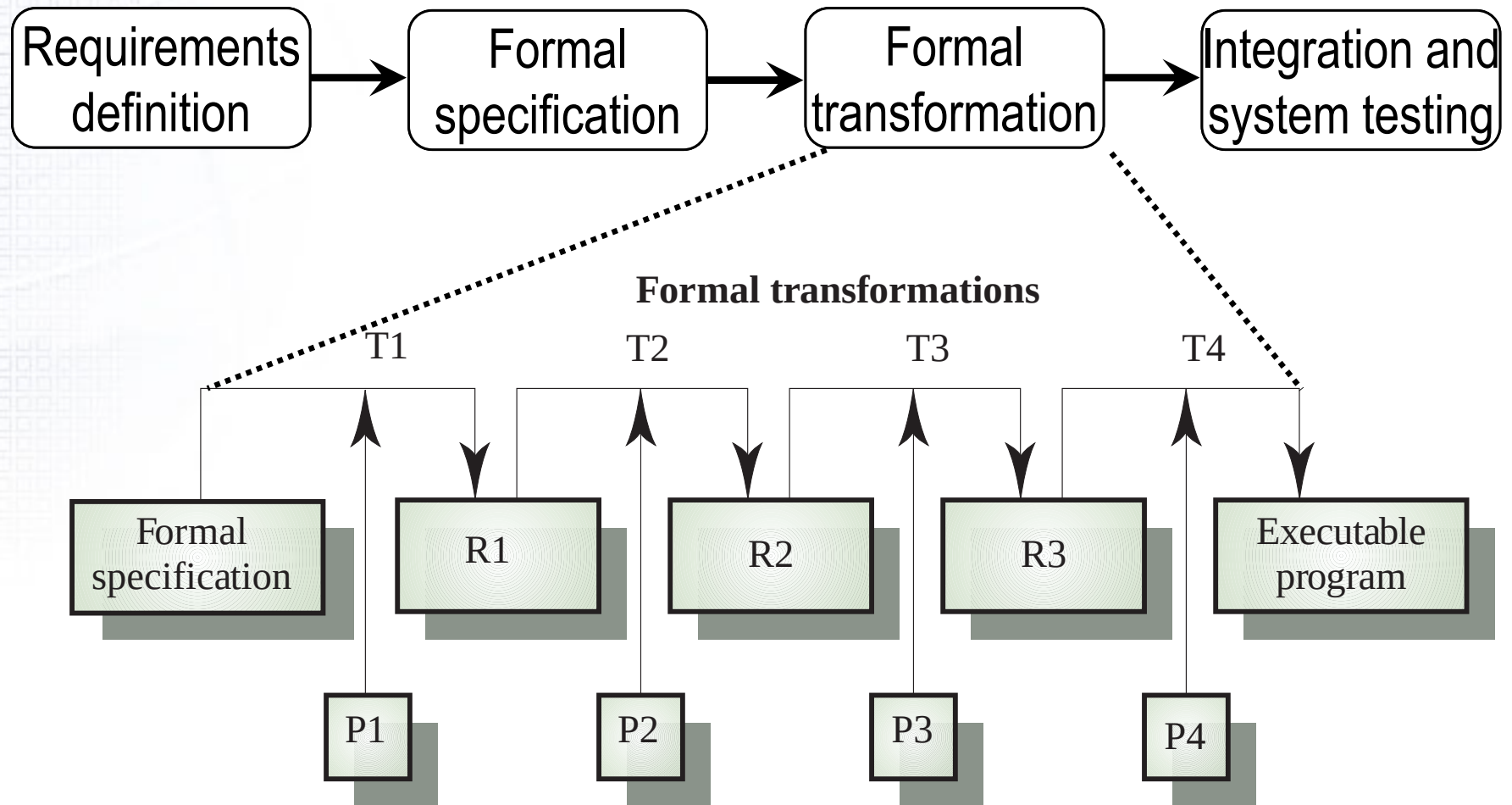
3.4 其它模型系列(3/5)





3.4 其它模型系列(4/5)

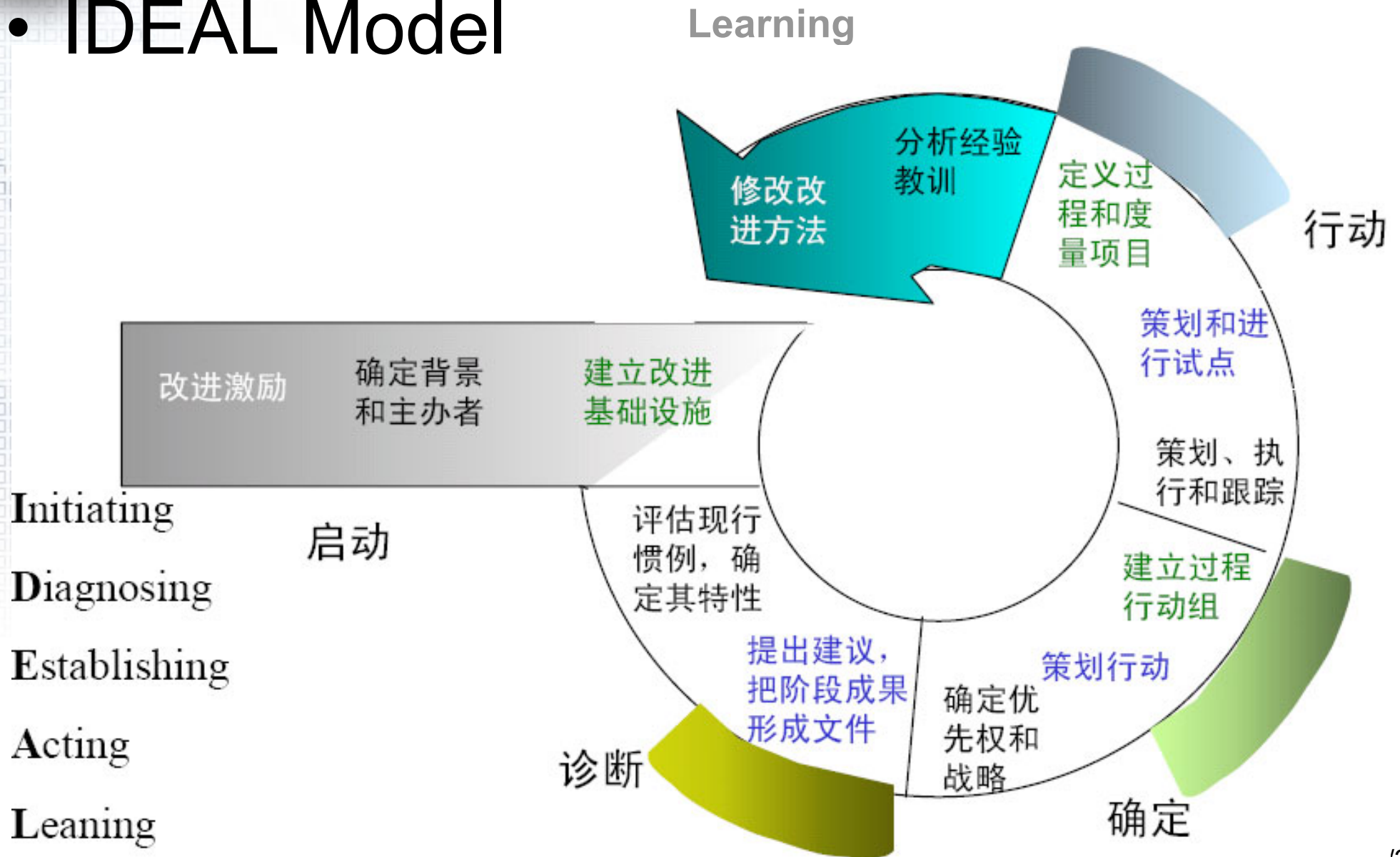
- 形式化方法模型





3.4 其它模型系列(5/5)

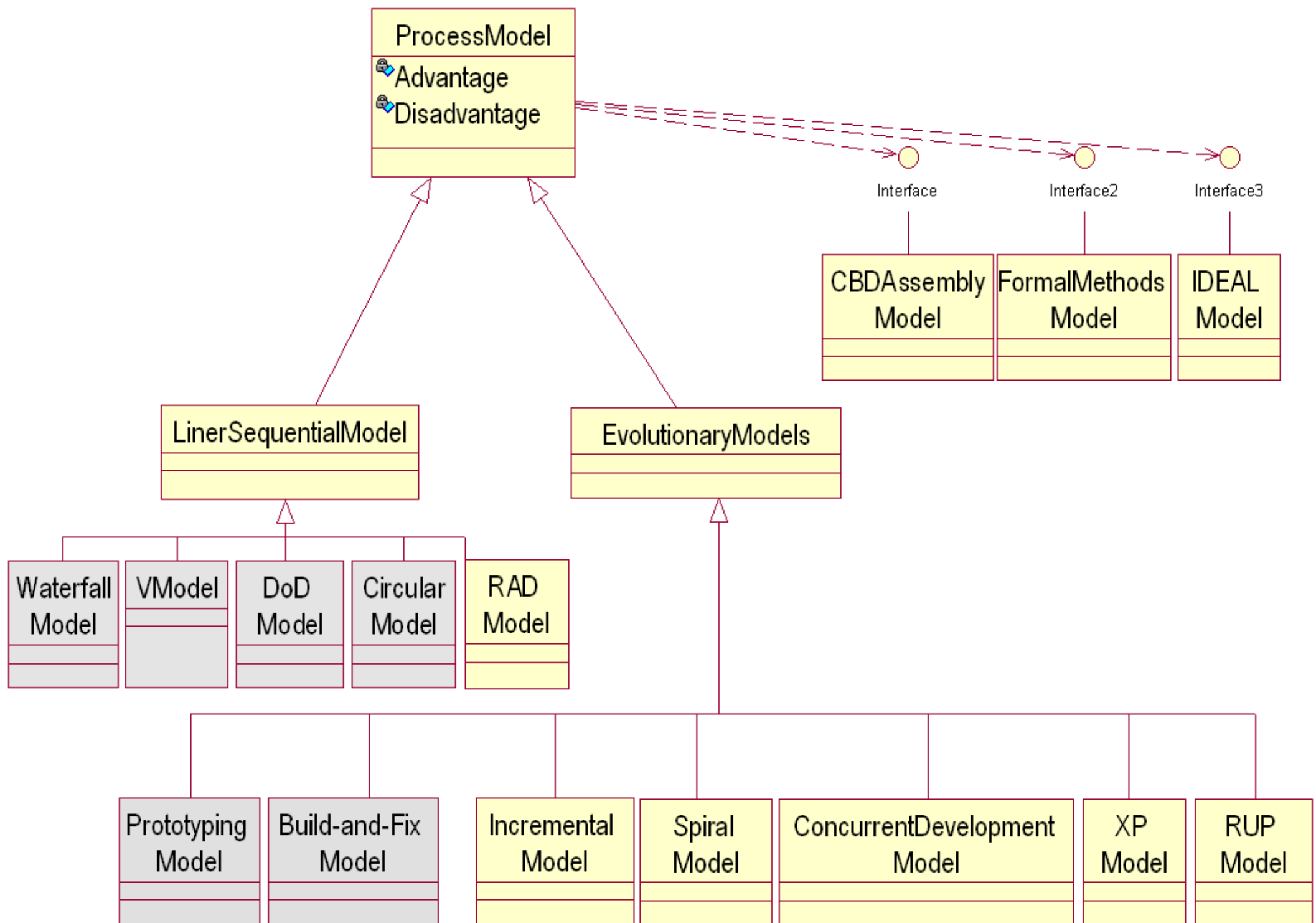
- IDEAL Model





3.5 过程模型的选择

- 软件工程过程模型的选择是基于：
 - 项目的应用特点
 - 背景熟悉？软件规模？时间紧？需求明确？
 - 采用的方法和工具
 - 传统？OO？
 - 需要的控制
 - 人员充裕？需求经常变化吗？架构？
 - 交付的产品
 - 允许增量的交付？（3月？6月？1年？）
 - 项目风险程度
 - 需要强调风险分析和管理的模型
 - 软件质量的要求
 - 需要严格和验证过程





敏捷过程

- 2001年2月
- 在犹他州瓦萨奇山的滑雪胜地
- 17位软件开发领域领军人物聚在一起，分享、探讨了非常多的软件开发方式、经验
- Agile Process
- 制定并发布《敏捷宣言》



敏捷宣言

敏捷宣言

我们一直在实践中探寻更好的软件开发方法，身体力行的同时也帮助他人。由此我们建立了如下价值观：

个体和互动 高于 流程和工具
工作的软件 高于 详尽的文档
客户合作 高于 合同谈判
响应变化 高于 遵循计划

也就是说，尽管右项有其价值，我们更重视左项的价值



敏捷宣言的12条原则

- 1. 目的：我们最重要的目标，是通过持续不断地及早交付有价值的软件使客户满意。**尽快交付**
- 2. 态度：乐于接受需求变更，即使是在开发后期也应如此。敏捷过程能够驾驭变化，从而为客户赢得竞争优势。**拥抱变化**
- 3. 关注：频繁交付可使用的软件，交付间隔越短越好，周期可以是几星期到几个月不等**持续交付**
- 4. 合作：在整个项目开发期间，业务人员和开发人员必须朝夕工作在一起。**集中办公**



敏捷宣言的12条原则

- 5.核心：围绕那些有推动力的人们来构建项目。给予他们所需的环境和支持，并且信任他们能够把工作完成好。

正向激励

- 6.沟通：与开发团队以及在开发团队内部最快速、有效的传递信息的方法就是，面对面的交谈。

面对面沟通

- 7.标准：可使用的软件是进度的主要衡量指标。

成果衡量进度

- 8.倡导：敏捷过程提倡可持续发展。出资人、开发人员以及使用者应该总是共同维持稳定的开发速度。

可持续作业



敏捷宣言的12条原则

- 9.追求：为了增强敏捷能力，应持续关注技术上的杰出成果和良好的设计。持续改进
- 10.根本：简洁——最大化不必要工作量的艺术——是至关重要的。简洁工作
- 11.团队：最好的架构、需求和设计都源自自我组织的团队。自组织团队
- 12.调整：团队应该定期反思如何能变得更有战斗力，然后相应地转变并调整其行为。定期总结



敏捷方法的特点

- 小和少

- 生成少量软件文档，每个文档规模要小
- 每次迭代要实现软件功能的数量和规模要小，迭代周期要小

- 简

- 技术、工具以及每次迭代要解决的问题尽可能简单
- 只关注当前欲实现的功能需求，而不要考虑将来的问题

- 快

- 快速响应变化、从用户处获得反馈，给用户提交有价值软件，对软件产品进行迭代和更新

- 变

- 允许需求动态变化，要以变应变，开发团队应是自组织的



用户故事

用户故事

定义：用户故事描述了对用户、系统或软件购买者有价值的功能。

三个要素：

角色

谁要使用这个功能。

功能

需要完成什么样的功能。

价值

为什么需要这个功能，这个功能带来什么样的价值。



- 作为一个<角色>, 我想要<功能>, 以便于<商业价值>



用户故事-案例

• 系统管理员、报账接待员、项目经理、故障处理员

0-a.作为系统管理员,我希望可以登录系统,以便我可以使用系统。

a.我希望可以验证我的登录信息,以便防止非本人登录我的账户。

b.我希望可以设置保密信息,以便我可以及时找回丧失的密码。

0-b.作为系统管理员,我希望可以增删改查系统用户的信息,以便我管理系统 的用户。

a.我希望可以增加账户信息,以便可以让新的人员可以使用系统。

b.我希望可以删除账户信息,以便防止账户被非法利用。

c.我希望可以修改账户信息,以便可以让该账户的信息得到更新。

d.我希望可以查询账户信息,以便可以了解指定的账户信息。

0-c.作为系统管理员,我希望可以审批注册的账户信息,以便我管理系统的用 a.我希望可以审批注册的账户信息,以便防止非法用户注册账号使用系统。 b.我希望可以账户信息包含用户的身份信息,以便我管理系统的用户。

0-d.作为系统管理员,我希望拥有其他用户的所有权限,以便我监控系统的工作状态。

0-e.作为系统管理员,我希望可以查看登录记录,以便我监控系统的安全性。

敏捷开发流程图

